

Stack & Linked List

Monotonic stack, pointer manipulation – pointers మీద పట్టు ఇక్కడే వస్తుంది.

ప్రతి problem కి: ఏ **pattern** ఇది → ఎందుకు ఆ pattern → dry run → optimal JavaScript code → complexity → edge cases. [DSA_Patterns_Telugu.pdf](#)
pattern-first దృష్టి; ఈ file ఆ patterns ని నిజమైన LeetCode problems మీద అమలు చేస్తుంది.

ఈ document చదివిన తర్వాత Stack మరియు Linked List problems మళ్ళీ జీవితంలో మర్చిపోకూడదు. ప్రతి problem కి – ఎలా ఆలోచించాలి (intuition first), ఒక vivid real-life analogy, naive నుండి optimal వరకు thought process, clean commented JavaScript solution, పెన్సిల్తో గీసినట్టు dry run, complexity reasoning, pattern takeaway, మరియు edge cases (null, single node, cycles) – అన్నీ ఉంటాయి.

లక్ష్యం: DSA అస్సలు తెలియని person ని – ఎవరికైతే stack అంటే ఏమిటో, pointer అంటే ఏమిటో కూడా తెలియదో – వాళ్ళని SSE (Senior Software Engineer) interview లో confident గా ఈ 16 problems solve చేసేలా తయారు చేయడం. మనం facts బట్టి పట్టడం కాదు – **ఎలా ఆలోచించాలో** నేర్చుకుంటాం. Pattern అర్థమైతే, ఈ 16 మాత్రమే కాదు, వీటిలాంటి 100 problems కూడా solve చేయగలవు.

గమనిక: Big-O notation, arrays, recursion, hash map basics, time/space complexity అంటే ఏమిటి – ఇలాంటి పునాదులు (fundamentals) [DSA_00_Foundations_Telugu.md](#) లో ఉన్నాయి. అవి ముందు చదివితే ఈ guide ఇంకా సులభంగా అర్థమవుతుంది. ఇక్కడ మనం నేరుగా Stack & Linked List patterns లోకి దిగుతాం.

విషయ సూచిక (Table of Contents)

Pattern Primers (ముందు వీటిని చదువు – problems కి foundation)

- Pattern: Stack (LIFO, matching/parsing, monotonic stack, expression evaluation)
- Pattern: Linked List techniques (ListNode, traversal, dummy head, fast & slow, in-place reverse, two-pass)

Part 1 – Stack Problems

1. Valid Parentheses (LeetCode #20) – Easy
2. Simplify Path (LeetCode #71) – Medium
3. Min Stack (LeetCode #155) – Medium
4. Evaluate Reverse Polish Notation (LeetCode #150) – Medium
5. Basic Calculator (LeetCode #224) – Hard

Part 2 – Linked List Problems

6. Linked List Cycle (LeetCode #141) – Easy
7. Add Two Numbers (LeetCode #2) – Medium
8. Merge Two Sorted Lists (LeetCode #21) – Easy
9. Copy List with Random Pointer (LeetCode #138) – Medium
10. Reverse Linked List II (LeetCode #92) – Medium
11. Reverse Nodes in k-Group (LeetCode #25) – Hard
12. Remove Nth Node From End of List (LeetCode #19) – Medium

13. Remove Duplicates from Sorted List II (LeetCode #82) – Medium

14. Rotate List (LeetCode #61) – Medium

15. Partition List (LeetCode #86) – Medium

16. LRU Cache (LeetCode #146) – Medium

Pattern: Stack

వివరణ

Stack అంటే ఒక data structure – దీనిలో మనం **ఒక చివర నుండి మాత్రమే** items add చేయగలం, తీయగలం. ఆ చివరని **top** అంటారు. అంటే: చివరిగా లోపలికి వెళ్ళిన item **మొదట** బయటికి వస్తుంది. దీన్నే **LIFO** (Last In, First Out) అంటారు.

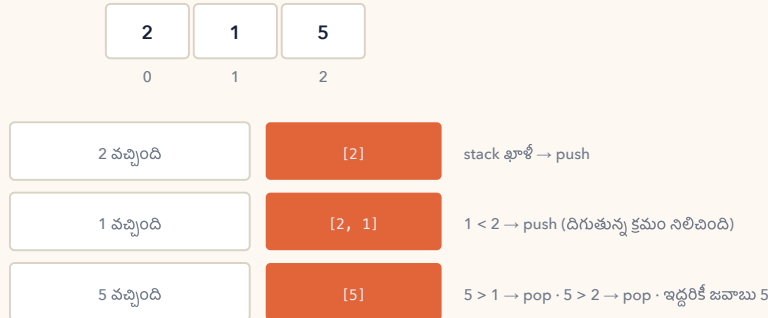
మూడు core operations ఉంటాయి, అన్నీ **$O(1)$** (constant time):

- **push(x)** – top మీద x పెట్టు.
- **pop()** – top item తీసేయి, దాన్ని return చెయ్యి.
- **peek() / top()** – top item ఏమిటో చూడు (తీయకుండా).

JavaScript లో ప్రత్యేకంగా Stack class అవసరం లేదు – **plain array** ని stack లా వాడతాం. `arr.push()` = push, `arr.pop()` = pop, `arr[arr.length - 1]` = peek. ఈ మూడూ amortized $O(1)$.

MONOTONIC STACK · NEXT GREATER / SMALLER FAMILY

MONOTONIC STACK · "తర్వాతి పెద్ద element" – [2, 1, 5]



ప్రతి element ఒకసారి push, ఒకసారి pop

అందుకే nested loop లా కనిపించినా మొత్తం $O(n)$.
ఈ amortised వాదనని interview lo స్పష్టంగా చెప్పాలి –
లేకపోతే interviewer $O(n^2)$ అనుకుంటాడు.

Stack lo ఏ క్రమం ఉంచాలి: "next greater" కావాలంటే దిగుతున్న stack · "next smaller" కావాలంటే పెరుగుతున్న stack. Daily Temperatures, Largest Rectangle, Stock Span – అన్నీ ఇదే.

Real-life Scenario

ఒక **hotel లో plates stack** ని ఊహించుకో. కొత్త plate కడిగి తెస్తే, మిగతా వాటి పైన పెడతారు. Customer కి plate కావాలంటే, **పైనున్నది** తీస్తారు – కింద ఉన్న మొదటి plate కి ఎప్పటికీ చెయ్యి చేరదు, పైవన్నీ తీసేదాకా. చివరిగా పెట్టిన plate మొదట బయటికి వెళ్తుంది. అదే LIFO.

ఇంకో analogy: **browser లో Back button**. నువ్వు $A \rightarrow B \rightarrow C$ pages visit చేస్తే, Back నొక్కితే C నుండి B కి వెళ్తావు (చివరిగా visit చేసినది మొదట undo). ప్రతి "undo/back/matching/nesting" situation వెనుక stack ఉంటుంది.

ఎలా గుర్తించాలి (Recognition Signals)

ఈ signals కనిపిస్తే stack గురించి ఆలోచించు:

- **Matching / nesting** – brackets `()[]{}`, tags, "valid/balanced" అనే మాట. లోపలిది ముందు close అవ్వాలి $\rightarrow$ LIFO.
- **"చివరిది ముందు కావాలి"** – undo, back, redo, most-recent.
- **Parsing / expression evaluation** – RPN, calculator, `../` path resolution. ఒక్కో token process చేస్తూ, పాత context ని గుర్తుంచుకోవాలి.
- **Nested structure ని "unwind" చేయడం** – parentheses లోపల parentheses, recursion ని iterative గా మార్చడం.
- **Monotonic Stack** – "next greater element", "next smaller", "కుడివైపు ఎత్తైన element" – increasing/decreasing order maintain చేసే special stack. (ఈ guide లో direct problem లేదు, కానీ idea తెలుసుకో: stack లో ఎప్పుడూ increasing (లేదా decreasing) sequence మాత్రమే ఉంచుతూ, order break అయ్యే element వచ్చినప్పుడు pop చేస్తూ answer లెక్కిస్తాం. Histogram, temperatures లాంటి problems కి కీలకం.)

Template Code (JavaScript)

```
// JS లో array నే stack లా వాడతాం – ప్రత్యేక class అవసరం లేదు.
const stack = [];

stack.push(10);           // [10]           -> top మీద పెట్టు
stack.push(20);           // [10, 20]
const top = stack[stack.length - 1]; // 20 -> peek (తీయకుండా చూడు)
const out = stack.pop();  // out = 20, stack = [10] -> top తీసియి
const isEmpty = stack.length === 0; // ఖాళీగా ఉందా?

// ⚠️ జాగ్రత్త: ఖాళీ array మీద pop() చేస్తే `undefined` వస్తుంది (error రాదు).
// కాబట్టి pop చేసే ముందు length చెక్ చేయడం మంచి అలవాటు.

// --- Monotonic stack idea (increasing stack skeleton) ---
function nextGreaterElements(nums) {
  const res = new Array(nums.length).fill(-1);
  const stack = []; // ఇక్కడ INDICES store చేస్తాం (values కాదు)
  for (let i = 0; i < nums.length; i++) {
    // ప్రస్తుత element, stack top మీద ఉన్న వాటికన్నా పెద్దదైతే,
    // వాటన్నిటికీ "next greater" ఇదే -> pop చేస్తూ answer నింపు.
    while (stack.length && nums[i] > nums[stack[stack.length - 1]]) {
      res[stack.pop()] = nums[i];
    }
  }
}
```

```

    }
    stack.push(i);
}
return res; // stack లో మిగిలిన వాటికి next greater లేదు → -1 అలానే ఉంటుంది
}

```

Complexity

- **Push / Pop / Peek:** $O(1)$ each.
- **ఒక array ని stack తో ఒకసారి scan చెయ్యడం:** $O(n)$ time. ప్రతి element ఎక్కువలో ఎక్కువ ఒకసారి push, ఒకసారి pop అవుతుంది – అందుకే monotonic stack కూడా $O(n)$, లోపల while ఉన్నా సరే (amortized analysis).
- **Space:** $O(n)$ worst case (అన్ని elements stack లోకి వెళ్తే).

1. Valid Parentheses (LeetCode #20) – Easy

సమస్య: ఒక string `s` ఇస్తారు, అందులో `(, ), {, }, [, ]` characters మాత్రమే ఉంటాయి. ఈ string **valid** అవునా కాదా చెప్పాలి. Valid అంటే:

1. ప్రతి open bracket, **సరైన రకం** close bracket తో close అవ్వాలి `( ( కి ) , [ కి ] , { కి } )`.
2. Brackets **సరైన order** లో close అవ్వాలి – లోపల తెరిచింది లోపలే మూయాలి.
3. ప్రతి close bracket కి, దానికి match అయ్యే open bracket ముందు ఉండాలి.

Constraints: `1 <= s.length <= 10^4`; `s` లో పైన చెప్పిన 6 characters మాత్రమే.

ఉదాహరణ:

```

Input: s = "()[]{}" → Output: true
Input: s = "]" → Output: false (రకం match కాలేదు)
Input: s = "([)]" → Output: false (order తప్పు – లోపల ( ముందు మూయాలి)
Input: s = "{[]}" → Output: true (సరిగ్గా nested)

```

ఎలా ఆలోచించాలి:

మొదట నీ చెయ్యి పెన్సిల్ తో string మీద పెట్టి, ఎడమ నుండి కుడికి చదువు. `( [ ] )` చూడు. `(` తెరిచావు. తర్వాత `[` తెరిచావు – ఇప్పుడు **చివరిగా తెరిచింది** `[` . తర్వాత `)` వచ్చింది. కానీ చివరిగా తెరిచింది `[` , `(` కాదు! కాబట్టి mismatch → invalid.

గమనించు: "**చివరిగా తెరిచింది**" అనేదే కీలకం. ఒక close bracket వచ్చినప్పుడు, అది **అత్యంత ఇటీవల తెరిచిన (most recent unmatched)** open bracket తో match అవ్వాలి. "చివరిది ముందు" = **LIFO** = **Stack**! ఇదే insight.

కాబట్టి plan: open bracket కనిపిస్తే stack లో push చెయ్యి. Close bracket కనిపిస్తే, stack top ని pop చేసి, అది సరైన matching open bracket చూడు. కాకపోతే invalid.

Brute Force / naive:

Stack ఆలోచన రాకపోతే, జనం string లో ఉన్న `()`, `[]`, `{}` జతలను పదేపదే వెతికి తీసేస్తారు: string లో `"()` ఉందా? ఉంటే తీసేయి. అలా repeatedly చేస్తూ, చివరికి string ఖాళీ అయితే valid, లేకపోతే invalid.

```
// Naive – repeated string replace
function isValidNaive(s) {
  let prev;
  do {
    prev = s;
    s = s.replace("()", "").replace("[]", "").replace("{} ", "");
  } while (s !== prev); // ఇంకా తీయలేనంత వరకు
  return s.length === 0;
}
```

- **Time:** $O(n^2)$ – ప్రతి `replace` pass $O(n)$, అలాంటి passes $O(n)$ అవ్వచ్చు (`"((((...)))")` లాంటి deep nesting కి).
- **Space:** $O(n)$ కొత్త strings కోసం.
- **ఎందుకు సరిపోదు:** $n = 10^4$ కి $O(n^2) = 10^8$ operations – నెమ్మది, పైగా strings పదేపదే copy అవ్వడం waste. ఒకే pass లో $O(n)$ లో solve చెయ్యొచ్చు.

Optimal Approach:

Insight: Close bracket ఎప్పుడూ **most-recent unmatched open bracket** తో match అవ్వాలి → Stack (LIFO). ఒకే pass:

1. ఒక stack తీసుకో.
2. ప్రతి character కి:
 - **Open bracket** అయితే → stack లో push.
 - **Close bracket** అయితే → stack ఖాళీగా ఉంటే invalid (match చేయడానికి ఏమీ లేదు). లేకపోతే pop చేసి, pop అయిన open bracket ఈ close కి సరైన జతా చూడు. కాకపోతే invalid.
3. String అయిపోయాక stack **ఖాళీగా** ఉంటే valid (అన్ని open brackets మూసేశాం). ఏమైనా మిగిలితే invalid (కొన్ని తెరిచి మూయలేదు).

Close → open mapping ని ఒక object లో పెడితే code శుభ్రంగా ఉంటుంది.

Solution (JavaScript):

```
function isValid(s) {
  const stack = [];
  // close bracket -> దానికి కావాల్సిన open bracket
  const pairs = { "(": ")", "[": "]", "{": "}" };

  for (const ch of s) {
    if (ch === "(" || ch === "[" || ch === "{") {
      // open bracket -> గుర్తుంచుకోవడానికి stack లో పెట్టు
      stack.push(ch);
    } else {
      // close bracket -> top ఉన్న open bracket ఇదే జతా చూడు.
      // stack ఖాళీ అయితే pop() = undefined, అది pairs[ch] కి match కాదు → false.
      if (stack.pop() !== pairs[ch]) {
        return false;
      }
    }
  }
}
```

```

    }
  }
}

// అన్నీ సరిగ్గా జత అయితే stack ఖాళీగా ఉండాలి.
return stack.length === 0;
}

```

Dry Run: `s = "([])"`

```

stack = []
ch = '(' : open -> push      stack = ['(']
ch = '[' : open -> push      stack = ['(', '[']
ch = ']' : close -> pop()='[' , pairs['']]='[' ; '[' !== '(' -> return false ✅

```

`s = "{[]}"` (valid case):

```

stack = []
ch = '{' : push      stack = ['{']
ch = '[' : push      stack = ['{', '[']
ch = ']' : pop()='[' , pairs['']]='[' ; match ✓ stack = ['{']
ch = '}' : pop()='{' , pairs['']]='{' ; match ✓ stack = []
loop ముగిసింది; stack.length === 0 -> return true ✅

```

Complexity:

- **Time:** $O(n)$ – ప్రతి character ఒకసారి process; push/pop $O(1)$.
- **Space:** $O(n)$ – worst case `"((((("` లాంటిది అయితే అన్ని characters stack లోకి వెళ్తాయి.

గుర్తుంచుకోవాల్సినది:

"Matching / nesting / balanced" అనే మాట వినగానే **Stack** అనుకో. లోపలిది ముందు మూయాలి = LIFO.

ఇదే pattern వాడే ఇతర problems: valid parentheses variants, **remove invalid parentheses**, **minimum add to make valid**, **HTML/XML tag matching**, **decode nested strings**

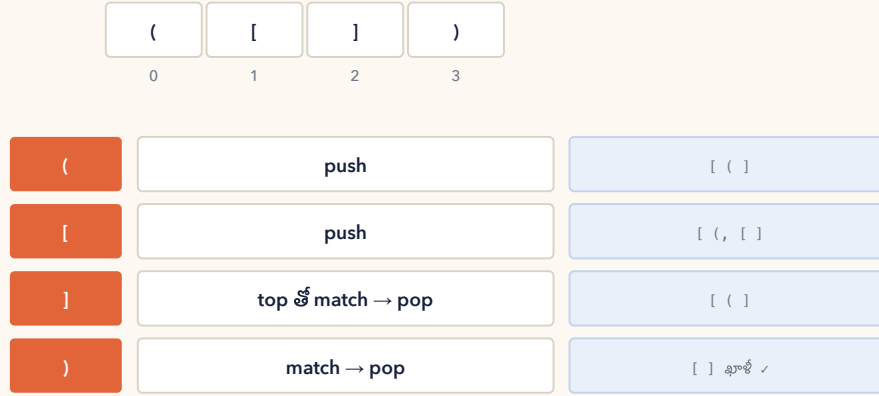
(`"3[a2[c]]"`), **basic calculator** (ఈ guide #5). రెండు కీలక checks మర్చిపోకు: (1) close వచ్చినప్పుడు stack ఖాళీగా ఉందా, (2) చివర్లో stack ఖాళీగా ఉందా.

సాధారణ తప్పులు:

- **చివర్లో stack empty చేక్ మర్చిపోవడం:** `"("` కి loop లో ఏ mismatch రాదు, కానీ stack లో `()` మిగిలిపోతుంది → చివర్లో `stack.length === 0` చూడకపోతే తప్పుగా `true` వస్తుంది.
- **రకం (type) match చూడకపోవడం:** కేవలం "open/close counts సమానమా" చూస్తే `"[]"` కి తప్పుగా valid అంటుంది. బ్రాకెట్ రకం కూడా match అవ్వాలి.
- **ఖాళీ stack మీద pop:** పైన code లో `stack.pop()` ఖాళీగా ఉంటే `undefined` return చేస్తుంది, అది ఏ open bracket కీ equal కాదు కాబట్టి safe గా `false` వస్తుంది – కానీ ఈ subtlety తెలియకపోతే crash అవుతుందని భయపడతారు. తెలుసుకో.
- **Odd length:** `s.length` బేసి సంఖ్య అయితే ఎప్పటికీ valid కాదు – కావాలంటే మొదట్లోనే `if (s.length % 2 !== 0) return false;` పెట్టి కొంచెం optimize చెయ్యొచ్చు (mandatory కాదు).

VALID PARENTHESES · LIFO ఎందుకు సరిపోతుంది

VALID PARENTHESES – stack ఎందుకు సరైన సాధనం



ఎందుకు stack

చివరిగా తెరిచినది మొదట మూయాలి – అదే LIFO. Stack యొక్క నిర్వచనమే ఇది.
చివర్లో stack ఖాళీ కాకపోతే – మూయని brackets మిగిలాయి → invalid.

2. Simplify Path (LeetCode #71) – Medium

సమస్య: Unix-style **absolute path** ఒకటి string గా ఇస్తారు (ఎప్పుడూ / తో మొదలవుతుంది). దాన్ని **canonical (simplified) form** లోకి మార్చాలి. నియమాలు:

- . అంటే **ప్రస్తుత directory** – దీన్ని విస్మరించు (ignore).
- .. అంటే **ఒక level పైకి (parent directory)** వెళ్ళు.
- బహుళ slashes (// , ///) ఒకే / గా treat చెయ్యి.
- ఏ directory/file name (. , .. కాని ఏదైనా) అలానే ఉంచు.

Canonical path: (1) / తో మొదలవ్వాలి, (2) directories మధ్య ఒకే / , (3) చివర్లో / ఉండకూడదు (root / తప్ప), (4) . , .. ఉండకూడదు.

Constraints: $1 \leq \text{path.length} \leq 3000$; path లో English letters, digits, . , / , _ ఉంటాయి; ఇది valid absolute Unix path.

ఉదాహరణ:

Input: "/home/"	→ Output: "/home"
Input: "/../"	→ Output: "/" (root కి పైన ఏమీ లేదు)
Input: "/home//foo/"	→ Output: "/home/foo"
Input: "/a/./b/../../c/"	→ Output: "/c"

ఎలా ఆలోచించాలి:

Path అంటే directories sequence, / లతో separate చేసినది. కాబట్టి మొదట / దగ్గర **split** చేస్తే, తలో ముక్క (component) దొరుకుతుంది: "", "a", ".", "b", ".." ...

ఇప్పుడు ఒక్క component చూస్తూ వెళ్దాం. "a" వస్తే directory లోకి దిగుతున్నాం – గుర్తుంచుకో. "." వస్తే? **చివరిగా దిగిన directory నుండి పైకి** రావాలి – అంటే చివరిగా గుర్తుంచుకున్నది తీసేయాలి. "చివరిది తీసేయడం" = **pop** = **Stack**! ఇదే insight. Directories ని stack లో push చేస్తూ, "." వస్తే pop చేస్తూ పోతే, చివర్లో stack లో మిగిలేవే నిజమైన path.

Brute Force / naive:

Stack లేకుండా చేయాలంటే – string మీద పదేపదే "." వెతికి, దానికి ముందున్న directory ని కలిపి రెండింటినీ regex/replace తో తీసేస్తూ పోవాలి. ఇది messy, edge cases (consecutive ".", root దగ్గర "..") చాలా వస్తాయి, పైగా ప్రతి replace $O(n)$ → overall $O(n^2)$, bug-prone.

- **Time:** $O(n^2)$, **Space:** $O(n)$.
- **ఎందుకు సరిపోదు:** String ని పదేపదే rebuild చేయడం నెమ్మది, edge cases చేతిలో పట్టుకోవడం కష్టం. Stack తో ఒకే pass, clean.

Optimal Approach:

Insight: Path components ని stack తో process చెయ్యి – directory అయితే push, "." అయితే pop, "." / ఖాళీ అయితే skip.

Plan:

1. `path.split("/")` – components array వస్తుంది.
2. ప్రతి component `part` కి:
 - `""` (double slash వల్ల) లేదా `"."` → **skip** (ఏమీ చేయకు).
 - `"."` → stack ఖాళీగా లేకపోతే **pop** (root దగ్గర ".." వస్తే ఏమీ చేయకు – root కి పైన ఏమీ లేదు).
 - మిగతా అన్నీ (నిజమైన directory names) → **push**.
3. చివర్లో `"/" + stack.join("/")` – canonical path. Stack ఖాళీ అయితే ఇది `"/"` (root) అవుతుంది. చివర trailing slash రాదు, ఇదే మనకి కావాల్సింది.

Solution (JavaScript):

```
function simplifyPath(path) {
  const stack = [];
  // "/" దగ్గర విడగొట్టు. "/" ఉంటే మధ్యలో "" (ఖాళీ string) వస్తుంది.
  const parts = path.split("/");

  for (const part of parts) {
    if (part === "" || part === ".") {
      // ఖాళీ (double slash) లేదా "current dir" -> ఏమీ చేయకు
      continue;
    }
    if (part === "..") {
      // ఒక level పైకి -> చివరి directory తీసేయి (ఉంటేనే)
      if (stack.length > 0) stack.pop();
    } else {
      // నిజమైన directory name -> stack లోకి దిగు
      stack.push(part);
    }
  }
}
```

```
// "/a/b" లా కలుపు. stack ఖాళీ అయితే "/" (root) వస్తుంది.
return "/" + stack.join("/");
}
```

Dry Run: path = `"/a/./b/../../c/"`

```
split("/") → [ "", "a", ".", "b", "..", "..", "c", "" ]
stack = []
"" → skip           stack = []
"a" → push          stack = ["a"]
"." → skip           stack = ["a"]
"b" → push           stack = ["a", "b"]
".." → pop           stack = ["a"]
".." → pop           stack = []
"c" → push           stack = ["c"]
"" → skip            stack = ["c"]
result = "/" + "c" = "/c" ✓
```

path = `"/./"` → split = `[ "", "..", "" ]` → `..` వచ్చినప్పుడు stack ఖాళీ, pop చేయము → stack = `[]` → result = `"/"` ✓

Complexity:

- **Time:** $O(n)$ – split $O(n)$, ప్రతి component ఒకసారి process, చివర join $O(n)$.
- **Space:** $O(n)$ – stack + split array.

గుర్తుంచుకోవాల్సినది:

"State ని maintain చేస్తూ, తర్వాత వచ్చే token పాతదాన్ని undo చేయగలదు" – ఇది stack signal. ఇక్కడ `..` పాత directory ని undo చేస్తుంది. మొదట **tokenize (split)**, తర్వాత **process each token with a stack** – ఇదే parsing pattern. Backspace string compare (`"ab#c"`), text editor undo, folder navigation అన్నీ ఇదే idea.

సాధారణ తప్పులు:

- **Root దగ్గర `..` :** stack ఖాళీగా ఉంటే pop చేయకూడదు (`if (stack.length > 0)`) మర్చిపోతే empty array మీద pop → `undefined`, logic పాడవుతుంది). `/../` → `/` రావాలి.
- **Double slash `//` :** split చేస్తే మధ్యలో `""` వస్తుంది – దాన్ని skip చేయాలి, లేకపోతే ఖాళీ directory push అవుతుంది.
- **Trailing slash:** `join("/")` వాడితే చివర slash రాదు – manually `+ "/"` కలపాద్దు.
- **"..." ని `..` గా పొరపడటం:** మూడు చుక్కలు (`...`) ఒక **valid directory name** – పైకి వెళ్ళే command కాదు. Exact equality (`part === ".."`) చెక్ చెయ్యి, `startsWith` కాదు.

DUMMY NODE · HEAD యొక్క SPECIAL CASE ని తొలగించడం

DUMMY NODE – head మారే ప్రతిచోటా

Dummy లేకుండా – head ని delete చేయాలంటే



head ప్రత్యేక సందర్భం →
if (head == target) head = head.next

Dummy తో – అన్నీ ఒకేలా



ప్రతి node కి prev ఉంది –
special case లేదు

ఎక్కడ తప్పనిసరి

Remove Nth from End · Remove Duplicates · Merge Two Lists · Partition List

నియమం: head మారే అవకాశం ఉంటే – dummy వాడండి.

వివరణ

return dummy.next – అంతే.

3. Min Stack (LeetCode #155) – Medium

సమస్య: ఒక stack design చెయ్యి – ఇది push, pop, top ఇవ్వడంతో పాటు, **stack లో ప్రస్తుతం ఉన్న minimum element ని O(1) లో** ఇవ్వాలి. ఈ methods implement చెయ్యి:

- `push(val)` – val ని stack మీద పెట్టు.
- `pop()` – top element తీసేయి.
- `top()` – top element return చెయ్యి.
- `getMin()` – stack లో minimum element return చెయ్యి.

నాలుగూ **O(1)** లో జరగాలి (ఇదే challenge).

Constraints: $-2^{31} \leq val \leq 2^{31} - 1$; `pop`, `top`, `getMin` ఎప్పుడూ non-empty stack మీదే call అవుతాయి; మొత్తం 3×10^4 calls వరకు.

ఉదాహరణ:

```
push(-2); push(0); push(-3);
getMin() → -3
pop();           // -3 తీసేశాం
top()           → 0
getMin() → -2    // ఇప్పుడు మిగిలినది {-2, 0}, min = -2
```

ఎలా ఆలోచించాలి:

`getMin()` కి O(1) కావాలి – అంటే minimum ని **ప్రతిసారీ లెక్కించకూడదు**, ముందే **తెలిసి ఉండాలి**. కానీ ఒక్క variable లో "current min" పెట్టుకుంటే సరిపోతుందా? ఆలోచించు: min element ని `pop` చేసేస్తే, **అంతకుముందు min ఏమిటి?** అది మనకు గుర్తు లేదు! ఒక్క variable చాలదు.

కీలక observation: stack **LIFO** – elements ఖచ్చితంగా push చేసిన reverse order లోనే pop అవుతాయి. కాబట్టి "ఈ element ఉన్నప్పుడు min ఎంత" అనేది **ప్రతి stack ప్లాయి** గుర్తుంచుకోగలం. Value push

చేసేటప్పుడే, "ఇప్పటివరకు min" ని పక్కన ఒక **రెండో stack** లో సమాంతరంగా (parallel) పెట్టు. Pop చేసేటప్పుడు రెండింటినీ కలిపి తీసేయి. అప్పుడు min-stack top ఎప్పుడూ ప్రస్తుత minimum.

Brute Force / naive:

ఒకే normal stack పెట్టుకుని, `getMin()` కి stack అంతా scan చేసి minimum వెతకడం.

```
getMin() {  
  let m = Infinity;  
  for (const x of this.stack) m = Math.min(m, x); // ప్రతిసారీ O(n)  
  return m;  
}
```

- **push/pop/top:** $O(1)$, కానీ **getMin:** $O(n)$.
- **ఎందుకు సరిపోదు:** Problem స్పష్టంగా **getMin $O(1)$** అడిగింది. 3×10^4 calls, ప్రతి getMin $O(n)$ అయితే మొత్తం $O(n^2) \rightarrow$ నెమ్మది, పైగా requirement violate.

Optimal Approach:

Insight: రెండో stack (min-stack) పెట్టు – దాని top ఎప్పుడూ "main stack లో ప్రస్తుత minimum". రెండింటినీ synchronize గా maintain చెయ్యి.

Plan:

1. రెండు stacks: `stack` (అసలు values), `minStack` (ఆ క్షణంలో min).
2. **push(val):** `stack` లో push. `minStack` ఖాళీగా ఉంటే, లేదా `val <= minStack top` అయితే, `minStack` లో కూడా `val` push. (`<=` వాడటం ముఖ్యం – duplicate min values ని సరిగ్గా handle చేయడానికి.)
3. **pop():** `stack` నుండి తీసేయి. తీసిన value `minStack top` కి **సమానం** అయితే, `minStack` నుండి కూడా pop (అది ఈ level కి min గా push అయింది).
4. **top():** `stack` top.
5. **getMin():** `minStack` top – ఇది $O(1)$.

ప్రత్యామ్నాయం (space optimization): ప్రతిసారీ min ని push చేయకుండా, min మారినప్పుడు మాత్రమే push చేసే variant, లేదా ఒకే stack లో `(value, minSoFar)` జతలు store చేసే variant కూడా ఉన్నాయి. ఇక్కడ చూపిన two-stack version అర్థం చేసుకోవడానికి, explain చేయడానికి సులభం – interview కి perfect.

Solution (JavaScript):

```
class MinStack {  
  constructor() {  
    this.stack = []; // అసలు values  
    this.minStack = []; // top = ఆ క్షణంలో minimum  
  }  
  
  push(val) {
```

```

    this.stack.push(val);
    // minStack ఖాళీ అయినా, లేదా కొత్త val ప్రస్తుత min కన్నా ≤ అయినా push.
    // "<=" వాడటం వల్ల ఒకే min value రెండుసార్లు ఉంటే రెండూ minStack లో ఉంటాయి.
    if (this.minStack.length === 0 || val <= this.getMin()) {
        this.minStack.push(val);
    }
}

pop() {
    const removed = this.stack.pop();
    // ఈ value ప్రస్తుత min అయితే, minStack నుండి కూడా తీసేయి.
    if (removed === this.getMin()) {
        this.minStack.pop();
    }
}

top() {
    return this.stack[this.stack.length - 1];
}

getMin() {
    return this.minStack[this.minStack.length - 1]; // O(1)
}
}

```

Dry Run: push(-2), push(0), push(-3), getMin, pop, top, getMin

```

push(-2): stack=[-2]           minStack empty → push   minStack=[-2]
push(0) : stack=[-2,0]         0 <= -2? no → skip    minStack=[-2]
push(-3): stack=[-2,0,-3]      -3 <= -2? yes → push   minStack=[-2,-3]
getMin(): minStack top = -3    ✓
pop()   : removed=-3; -3 === getMin()(-3)? yes → minStack.pop()
           stack=[-2,0]           minStack=[-2]
top()   : stack top = 0        ✓
getMin(): minStack top = -2    ✓

```

Complexity:

- **Time:** push / pop / top / getMin అన్నీ **O(1)**.
- **Space:** O(n) – worst case (values descending order లో వస్తే, e.g. 5,4,3,2,1) minStack కూడా n elements – main stack తో సమానం.

గుర్తుంచుకోవాల్సినది:

"O(1) లో aggregate (min/max) కావాలి" అంటే → ఆ aggregate ని **precompute చేసి, parallel structure లో store** చెయ్యి. ఇక్కడ "auxiliary stack" pattern. అదే idea తో **Max Stack, queue using stacks, stack తో O(1) getMax** వస్తాయి. కీలక takeaway: **stack యొక్క LIFO nature వల్ల, ప్రతి level కి కావాల్సిన extra info (min) ని ఆ level తోనే bind చేసి పెట్టుకోవచ్చు.**

సాధారణ తప్పులు:

- < బదులు <= వాడకపోవడం: push(2), push(2) అనుకో. < వాడితే రెండో 2 minStack లోకి రాదు (minStack= [2]). తర్వాత pop() చేస్తే 2 === getMin()(2) → minStack pop → minStack ఖాళీ!

కానీ main stack లో ఇంకా 2 ఉంది – min తప్పు. `<=` వాడితే duplicate min కూడా push అవుతుంది, ఈ bug రాదు.

- **pop లో min-stack update మర్చిపోవడం:** ప్రతి pop కి "ఇది min-a?" చెక్ చేసి, అవునైతే minStack నుండి కూడా తీయాలి. మర్చిపోతే getMin పాత (తప్పు) min చూపిస్తుంది.
- **ఖాళీ పరిస్థితి:** మొదటి push కి `getMin()` call చేస్తే minStack ఖాళీ → `undefined`. అందుకే push లో `minStack.length === 0` check మొదట పెట్టాలి (short-circuit).
- **top() లో pop చేయడం:** `top()` element చూడాలి, తీయకూడదు. `this.stack.pop()` పొరపాటున వాడెద్దు – `this.stack[this.stack.length - 1]`.

4. Evaluate Reverse Polish Notation (LeetCode #150) – Medium

సమస్య: Reverse Polish Notation (RPN) లో ఉన్న ఒక expression, tokens array గా ఇస్తారు. దాన్ని evaluate చేసి integer result ఇవ్వాలి. RPN అంటే **postfix notation** – operator, తన రెండు operands తర్వాత వస్తుంది. Valid operators: `+`, `-`, `*`, `/`. మిగతా tokens integers.

రెండు integers మధ్య division **truncates toward zero** (అంటే `6 / -4 = -1`, `-1` కాదు `-2`).

Constraints: `1 <= tokens.length <= 10^4`; token ఏదైనా operator లేదా `[-200, 200]` range integer; expression ఎప్పుడూ valid; division by zero ఉండదు; answer 32-bit integer లో పడుతుంది.

ఉదాహరణ:

Input: ["2","1","+","3","*"]	→ Output: 9	అర్థం: ((2 + 1) * 3) = 9
Input: ["4","13","5","/","+"]	→ Output: 6	అర్థం: (4 + (13 / 5)) = 4 + 2 = 6

ఎలా ఆలోచించాలి:

మనం స్క్రూల్స్ నేర్చుకున్నది **infix**: `2 + 1`. RPN లో అదే `2 1 +` – operator చివర్లో. దీని అందం: **brackets అవసరం లేదు**, precedence confusion లేదు. మరి evaluate ఎలా?

Tokens ని ఎడమ నుండి చదువుతూ ఆలోచించు: number కనిపిస్తే దాన్ని పక్కన పెట్టుకో (ఇంకా ఏం చేయాలో తెలియదు). Operator (`+`) కనిపిస్తే, **అది అంతకుముందు వచ్చిన రెండు numbers మీద apply అవ్వాలి**. "అంతకుముందు వచ్చిన రెండు" – అంటే మనం పక్కన పెట్టుకున్న వాటిలో **చివరి రెండు**. చివరివి ముందు కావాలి = **Stack!** Numbers ని stack లో push చేస్తూ, operator వచ్చినప్పుడు రెండు pop చేసి, లెక్క చేసి, result తిరిగి push చెయ్యి. చివర్లో stack లో మిగిలిన ఒకే number = answer.

Brute Force / naive:

RPN ని infix గా మార్చి, brackets పెట్టి, precedence తో evaluate చేయడం – అనవసర complexity. లేదా repeatedly array లో "number number operator" pattern వెతికి, ఆ మూడింటినీ ఒక result తో replace చేస్తూ పోవడం:

```
// Naive - పదేపదే array లో మూడు tokens కుదించడం
// [2,1,+,3,*] → [3,3,*] → [9]. ప్రతి pass array మధ్యలో splice → O(n) shift.
```

- **Time:** $O(n^2)$ – ప్రతి reduction $O(n)$ splice, $n/2$ reductions.
- **ఎందుకు సరిపోదు:** Array మధ్యలో splice ఖరీదైనది (elements shift అవుతాయి). Stack తో single pass $O(n)$ లో అవుతుంది – RPN కి stack natural fit.

Optimal Approach:

Insight: RPN evaluation = stack. Number → push. Operator → రెండు pop, apply, push result.

Plan:

1. ఒక stack.
2. ప్రతి token:
 - **Number** → `Number(token)` గా push.
 - **Operator** → `b = pop()` (రెండేది = right operand), `a = pop()` (మొదటిది = left operand). `a op b` లెక్కించి push.
3. **Order జాగ్రత్త:** చివరిగా push అయినది `b` (right operand). `a - b`, `a / b` లో `a` మొదట – pop order ని తారుమారు చేయకు!
4. **Division:** `Math.trunc(a / b)` – JS లో `/` float ఇస్తుంది (`5/2 = 2.5`), పైగా `Math.floor(-1/...)` negative కి తప్పు (floor toward $-\infty$). `Math.trunc` toward zero → problem కి సరిపోతుంది.
5. చివర్లో `stack.pop()` = answer.

Solution (JavaScript):

```
function evalRPN(tokens) {
    const stack = [];
    const ops = new Set(["+", "-", "*", "/"]);

    for (const token of tokens) {
        if (ops.has(token)) {
            // operator: చివరి రెండు operands తీయి. Order ముఖ్యం!
            const b = stack.pop(); // right operand (చివరిగా push అయినది)
            const a = stack.pop(); // left operand
            let res;
            if (token === "+") res = a + b;
            else if (token === "-") res = a - b;
            else if (token === "*") res = a * b;
            else res = Math.trunc(a / b); // toward-zero truncation
            stack.push(res); // ఫలితం తిరిగి stack లోకి
        } else {
            // number: string ని number గా మార్చి push
            stack.push(Number(token));
        }
    }
}
```

```
    return stack.pop(); // చివర మిగిలిన ఒకే value = answer
}
```

Dry Run: ["4", "13", "5", "/", "+"]

```
stack = []
"4" : number → push      stack = [4]
"13" : number → push     stack = [4, 13]
"5" : number → push      stack = [4, 13, 5]
"/" : b=pop()=5, a=pop()=13 → trunc(13/5)=trunc(2.6)=2 → push  stack = [4, 2]
"+" : b=pop()=2, a=pop()=4 → 4+2=6 → push                      stack = [6]
return pop() = 6 ✓
```

Toward-zero చూద్దాం: $13/-5 = -2.6 \rightarrow \text{Math.trunc}(-2.6) = -2$ (zero వైపు), కానీ $\text{Math.floor}(-2.6) = -3$ (తప్పు). అందుకే `trunc`.

Complexity:

- **Time:** $O(n)$ – ప్రతి token ఒకసారి; push/pop $O(1)$.
- **Space:** $O(n)$ – worst case అన్నీ numbers అయితే (operators రాకముందు) stack లో $n/2$ దాకా.

గుర్తుంచుకోవాల్సినది:

Postfix expression evaluation = stack, ఒక్క pass. Operands ని hold చేస్తూ, operator వచ్చినప్పుడు "చివరి కొన్నింటిని" తీసి combine చేసే ఏ problem అయినా ఇదే pattern (RPN, **Basic Calculator II** $3+2*2$, **decode string**, function-call argument evaluation). రెండు కీలక జాగ్రత్తలు: **(1) pop order** (a op b లో a మొదట pop కాదు – b మొదట), **(2) integer division** rounding direction.

సాధారణ తప్పులు:

- **Operand order తారుమారు:** non-commutative ops (`-`, `/`) కి a మరియు b order తప్పితే తప్పు answer. `b = pop()` మొదట (right), `a = pop()` తర్వాత (left). `["5", "1", "-"] = 5 - 1 = 4`, `1 - 5 = -4` కాదు.
- **Math.floor వాడటం:** negative division కి floor toward $-\infty \rightarrow$ తప్పు. `Math.trunc` (toward zero) వాడు. లేదా `(a / b) | 0` (bitwise OR-0 కూడా toward-zero truncate చేస్తుంది, కానీ 32-bit range కి పరిమితం).
- **Number() మర్చిపోవడం:** tokens strings. `"2" + "3" = "23"` (string concat!), `2 + 3 = 5` కాదు. Push చేసేముందు `Number(token)`.
- **Negative numbers ని operator అనుకోవడం:** `"-11"` ఒక number, operator కాదు. `ops.has(token)` set-membership check ఈ pitfall ని దాటేస్తుంది (`"-"` set లో ఉంది కానీ `"-11"` లేదు). `token === "-"` బదులు ఏదైనా "మొదటి అక్షరం `-`" లాంటి check పెడితే bug.

5. Basic Calculator (LeetCode #224) – Hard

సమస్య: ఒక string `s` – ఒక valid arithmetic expression – ఇస్తారు. దాన్ని evaluate చేసి integer result ఇవ్వాలి. Expression లో ఉండేవి: **non-negative integers**, `+`, `-`, `(`, `)`, మరియు spaces. `*`, `/` లేవు (ఇది #224; multiplication/division ఉన్నది #227). కానీ **parentheses ఎన్ని levels అయినా nest** అవ్వచ్చు, unary minus `(-...)`, `-2` కూడా రావచ్చు.

Constraints: `1 <= s.length <= 3*10^5`; `s` valid; result 32-bit integer లో పడుతుంది. (`eval()` వాడకూడదు – అది cheating, పైగా interview లో reject.)

ఉదాహరణ:

Input: "1 + 1"	→ Output: 2
Input: " 2-1 + 2 "	→ Output: 3
Input: "(1+(4+5+2)-3)+(6+8)"	→ Output: 23
Input: "- (3 + (4 + 5))"	→ Output: -12 (unary minus)

ఎలా ఆలోచించాలి:

Parentheses లేకపోతే easy: ఎడమ నుండి చదువుతూ, running total కి `+num` లేదా `-num` కలుపుతూ పోతే సరిపోతుంది. అసలు కష్టం **parentheses**. `1 + (2 - 3)` లో, `(` వచ్చినప్పుడు మనం ఒక కొత్త **sub-problem** లోకి దిగుతున్నాం – కానీ **బయటి context** (ఇప్పటివరకు `total = 1`, మరియు `(` ముందున్న `sign = +`) ని మర్చిపోకూడదు. `)` వచ్చినప్పుడు ఆ బయటి context కి తిరిగి రావాలి.

"లోపలికి దిగినప్పుడు బయటి state గుర్తుంచుకో, తిరిగొచ్చినప్పుడు restore చెయ్యి" – ఇది **nesting**, ఇది **Stack!** `(` వచ్చినప్పుడు ఇప్పటివరకు `result` మరియు `sign` ని stack లో push చేసి, fresh గా మొదలుపెట్టు. `)` వచ్చినప్పుడు లోపలి `result` ని లెక్కించి, stack నుండి బయటి `sign`, బయటి `result` తీసి, `outerResult + outerSign * innerResult` గా కలుపు.

Brute Force / naive:

Recursion: `(` కనిపిస్తే matching `)` దాకా substring తీసి recursive call. Elegant గా అనిపిస్తుంది కానీ – matching `)` వెతకడం $O(n)$, substring copy $O(n)$, deep nesting కి stack overflow ప్రమాదం, పైగా pointer/index management గజిబిజి. (గమనిక: ఒకే pass recursion తో index ని reference గా carry చేస్తే $O(n)$ సాధ్యం, కానీ implement చేయడం iterative stack కన్నా error-prone.)

- **Time:** substring approach $O(n^2)$ worst case, **Space:** $O(n)$ recursion + copies.
- **ఎందుకు సరిపోదు:** `n = 3*10^5`. $O(n^2)$ time-out. Substring copies memory waste. Explicit stack తో ఒకే pass, $O(n)$, no recursion depth risk.

Optimal Approach:

Insight: ఒకే left-to-right pass. నాలుగు variables maintain చెయ్యి – `result` (ఇప్పటివరకు total), `sign` (తర్వాత వచ్చే number కి `+1/-1`), `num` (ప్రస్తుతం build అవుతున్న multi-digit number). `(` వచ్చినప్పుడు `result`, `sign` ని stack లో push చేసి reset. `)` వచ్చినప్పుడు pop చేసి combine.

Plan – ప్రతి character `ch`:

- **Digit** → `num = num * 10 + digit` (multi-digit numbers build చెయ్యి, e.g. "42" → 4, తర్వాత 42).
- **+** → పొగుచేసిన `num` ని apply: `result += sign * num; num = 0; sign = +1`.
- **-** → `result += sign * num; num = 0; sign = -1`.
- **(** → ఇప్పటి `result` ని stack లో push, తర్వాత `sign` ని push; `result = 0, sign = +1` (fresh sub-expression).
- **)** → `result += sign * num; num = 0`; తర్వాత `result *= stack.pop()` (బయటి sign), `result += stack.pop()` (బయటి result).
- **Space** → skip.

Loop అయ్యాక చివరి number apply: `result += sign * num`. Return `result`.

Solution (JavaScript):

```
function calculate(s) {
    let result = 0; // ఇప్పటివరకు మొత్తం
    let sign = 1; // తర్వాత వచ్చే number కి గుర్తు (+1 / -1)
    let num = 0; // ప్రస్తుతం build అవుతున్న (multi-digit) number
    const stack = []; // ప్రతి '(' దగ్గర: [బయటి result, బయటి sign] push

    for (const ch of s) {
        if (ch >= "0" && ch <= "9") {
            // multi-digit number: "123" → 1, 12, 123
            num = num * 10 + (ch.charCodeAt(0) - 48); // '0' = code 48
        } else if (ch === "+") {
            result += sign * num; // పేరుకున్న num ని apply
            num = 0;
            sign = 1;
        } else if (ch === "-") {
            result += sign * num;
            num = 0;
            sign = -1;
        } else if (ch === "(") {
            // బయటి context ని దాచు, sub-expression కి fresh start
            stack.push(result);
            stack.push(sign);
            result = 0;
            sign = 1;
        } else if (ch === ")") {
            result += sign * num; // '(' లోపలి చివరి number
            num = 0;
            result *= stack.pop(); // '(' ముందున్న sign తో గుణించు
            result += stack.pop(); // '(' ముందున్న result కి కలుపు
        }
        // ch === " " (space) → ఏమీ చేయకు
    }

    result += sign * num; // చివరి number (loop లో apply కాలేదు)
    return result;
}
```

Dry Run: `s = "(1+(4+5+2)-3)+(6+8)"` (expected 23)

```

'(' : push result,sign+1 → stack=[0,1]; result=0,sign=1
'1' : num=1
'+' : result=0+1*1=1; num=0; sign=+1
'(' : push 1,+1 → stack=[0,1,1,1]; result=0,sign=1
'4' : num=4
'+' : result=4; sign=+1; num=0
'5' : num=5
'+' : result=9; sign=+1; num=0
'2' : num=2
')' : result=9+1*2=11; num=0; result*=pop()(+1)=11; result+=pop()(1)=12
      stack=[0,1] → 12 అంటే (4+5+2)=11 ని బయటి "1+" లో కలిపాం
'-' : result=12; num=0; sign=-1
'3' : num=3
')' : result=12+(-1)*3=9; num=0; result*=pop()(+1)=9; result+=pop()(0)=9
      stack=[] → లోపలి బ్లాక్ మొత్తం = 9
'+' : result=9; sign=+1; num=0
'(' : push 9,+1 → stack=[9,1]; result=0,sign=1
'6' : num=6
'+' : result=6; sign=+1; num=0
'8' : num=8
')' : result=6+8=14; result*=pop()(+1)=14; result+=pop()(9)=23
loop ముగిసింది; result += sign*num (num=0) → 23 ✓

```

Complexity:

- **Time:** $O(n)$ – ప్రతి character ఒకసారి; push/pop $O(1)$.
- **Space:** $O(n)$ – worst case deeply nested `"((((...)))")` కి stack depth $n/2$ దాకా ప్రతి `(` కి రెండు entries).

గుర్తుంచుకోవాల్సినది:

Parentheses = context ని save/restore = Stack. "లోపలికి దిగినప్పుడు బయటి state గుర్తుంచుకో, తిరిగొచ్చినప్పుడు restore చెయ్యి" – ఇది calculator, nested JSON parser, `{{template}}` engines, decode string `("3[a2[c]]")` అన్నిటిలో ఒకటే. **`(` = push state, reset; `)` = pop state, merge.**
`* / /` కూడా ఉంటే (#227), operator precedence కోసం ఇంకో stack (లేదా `lastNum / lastSign` trick) కావాలి – కానీ core idea అదే.

సాధారణ తప్పులు:

- **Multi-digit numbers:** ఒక్కో digit ని విడిగా treat చేస్తే `"42"` = 4 అప్పుడు 2, తప్పు. `num = num*10 + digit` తో numbers build చెయ్యి, non-digit వచ్చినప్పుడే apply.
- **) దగ్గర pop order:** `sign` **చివరిగా** push అయింది కాబట్టి **మొదట** pop (గుణించడానికి), తర్వాత `result` pop (కలపడానికి). తారుమారు చేస్తే తప్పు.
- **చివరి number apply మర్చిపోవడం:** `"1+2"` లో 2 తర్వాత operator లేదు – loop లో apply కాదు. Loop బయట `result += sign * num` తప్పనిసరి.
- **Spaces:** `"2-1"` లో spaces ఉన్నాయి. ఏ branch కీ match కాకుండా skip అవ్వాలి (పైన code లో అలానే జరుగుతుంది – వాటికి `else` లేదు).
- **eval() వాడటం:** interview లో strict no. Injection risk, పైగా "నీకు parsing వచ్చా" అనే core skill ని test చేయలేవు.

Pattern: Linked List techniques

వివరణ

Linked List అంటే nodes గొలుసు (chain). ప్రతి **node** లో రెండు విషయాలు: (1) ఒక **value** (`val`), (2)

తర్వాతి node కి pointer (`next`). చివరి node యొక్క `next = null` (గొలుసు అంతం). మనం మొత్తం list ని ఒక reference – **head** (మొదటి node) – ద్వారా పట్టుకుంటాం.

Array vs Linked List: Array లో elements memory లో పక్కపక్కనే (contiguous), కాబట్టి `arr[i]` ని index తో $O(1)$ లో చేరవచ్చు. Linked List లో nodes memory లో ఎక్కడైనా చెల్లాచెదురుగా ఉంటాయి, `next` pointers మాత్రమే వాటిని కలుపుతాయి. కాబట్టి **random access లేదు** – 5వ node కావాలంటే head నుండి 5 hops నడవాల్సిందే ($O(n)$). కానీ **తెలిసిన node దగ్గర insert/delete $O(1)$** – pointers మార్చితే చాలు, array లా elements shift అవసరం లేదు.

First – ListNode definition & traversal (JavaScript)

```
// ఈ guide అంతటా ఇదే ListNode వాడతాం. LeetCode కూడా ఇదే ఇస్తుంది.
class ListNode {
  constructor(val = 0, next = null) {
    this.val = val; // ఈ node value
    this.next = next; // తర్వాతి node (లేకపోతే null)
  }
}

// --- List తయారుచేయడం: 1 → 2 → 3 → null ---
const head = new ListNode(1, new ListNode(2, new ListNode(3)));

// --- Traversal (list అంతా నడవడం) – అత్యంత ప్రాథమిక operation ---
let current = head; // head ని కదపాద్దు! వేరే pointer తో నడుపు
while (current !== null) { // null వచ్చేదాకా = list అంతం దాకా
  console.log(current.val); // ఈ node తో ఏదైనా చెయ్యి
  current = current.next; // తర్వాతి node కి "hop"
}

// 👉 బంగారు నియమం: 'head' ని ఎప్పుడూ కదపాద్దు – లేకపోతే list మొదలు పోగొట్టుకుంటావు.
// ఎప్పుడూ ఒక temporary pointer (current) తో traverse చెయ్యి.
```

LINKED LIST · DUMMY, FAST-SLOW, REVERSE

మూడు కీలక techniques

1 · DUMMY NODE – head మారే ప్రతివేటా



head ని delete చేయాల్సి వస్తే – dummy ఉంటే special case లేదు

2 · FAST & SLOW – మధ్య node, cycle



fast చివరికి చేరినప్పుడు slow సరిగ్గా మధ్యలో

3 · REVERSE – prev, curr, next

నాలుగు lines – క్రిమం తప్పకూడదు

`next = curr.next · curr.next = prev`

`prev = curr · curr = next`

మొదటి line లేకపోతే తర్వాతి node కి దారి పోతుంది, list అక్కడే తెగుతుంది

Real-life Scenario

Treasure hunt (నిధి వేట) ని ఊహించుకో. మొదటి chit (head) నీ చేతిలో ఉంది. దానిలో value ఉంది, పైగా "తర్వాతి chit ఎక్కడ" అనే clue (next) ఉంది. అక్కడికెళ్తే మళ్ళీ value + తర్వాతి clue. చివరి chit లో "ఇక clue లేదు" (next = null) – వేట అయిపోయింది.

కీలక పాఠం: నీకు **ప్రస్తుత chit మాత్రమే** కనిపిస్తుంది, మిగతావి కావు. 5వ chit కి **నేరుగా దూకలేవు** – ఒక్కొక్కటి follow అవ్వాల్సిందే. అలాగే, ఒక chit పోగొట్టుకుంటే (next reference కోల్పోతే), దాని తర్వాతి chits **అన్నీ శాశ్వతంగా పోతాయి** (memory leak / lost list). అందుకే pointers మార్చేటప్పుడు, "next" ని ముందు ఒక temp లో పట్టుకోవడం అలవాటు చేసుకో.

ఎలా గుర్తించాలి (Recognition Signals)

- Input **head of a linked list** గా వస్తే – obviously ఇదే topic.
- **"in-place, O(1) extra space"** అడిగితే – pointers rewire చెయ్యాలి, కొత్త list కట్టొద్దు.
- **"reverse", "reorder", "merge", "rotate", "remove nth from end"** – classic linked-list manipulations.
- **"cycle / loop ఉందా"** – fast & slow pointers.
- **"middle node కనుక్కో"** – fast & slow (fast రెట్టింపు వేగం).

Four core techniques (ఈ 4 తెలిస్తే 90% problems పడతాయి)

1. Dummy (sentinel) head node – "head కూడా మారొచ్చు/పోవచ్చు" అనే తలనొప్పి తీసేస్తుంది.

Head నే delete చేయాల్సి రావొచ్చు, లేదా కొత్త head రావొచ్చు. అప్పుడు head ని ప్రత్యేకంగా handle చేయడం bug-prone. Trick: అసలు head **ముందు** ఒక fake node (dummy) పెట్టు. అన్ని operations dummy నుండి చెయ్యి, చివర్లో dummy.next return చెయ్యి. ఇప్పుడు "real head" కూడా ఒక సాధారణ node లా, ప్రత్యేక case అవసరం లేదు.

```
const dummy = new ListNode(0, head); // dummy.next = అసలు head
let prev = dummy;
```

```
// ... prev ని కదిలిస్తూ nodes insert/delete చెయ్యి ...  
return dummy.next; // కొత్త (బహుశా మారిన) head
```

2. Fast & Slow pointers (Floyd's) – cycle detection, middle కనుక్కోవడం.

రెండు pointers: **slow** ఒక్కో step, **fast** రెండేసి steps. Cycle ఉంటే fast తప్పకుండా slow ని loop లోపల "చుట్టి" కలుస్తుంది (racetrack లో fast runner ని slow ని lap చేసినట్టు). Cycle లేకపోతే fast మొదట **null** చేరుతుంది. Middle కావాలంటే: fast చివరికి చేరేసరికి slow ఖచ్చితంగా మధ్యలో ఉంటుంది.

```
let slow = head, fast = head;  
while (fast !== null && fast.next !== null) {  
  slow = slow.next;      // 1 step  
  fast = fast.next.next;  // 2 steps  
  // if (slow === fast) → cycle!  
}  
// loop ముగిస్తే: slow = middle (cycle లేకపోతే)
```

3. In-place reversal – pointers ని వెనక్కి తిప్పడం, O(1) space.

మూడు pointers: **prev**, **curr**, **next**. ప్రతి step లో **curr.next** ని **prev** వైపు తిప్పు, తర్వాత **curr** ముందుకు ఒక అడుగు ముందుకు జరుపు. **next** ని ముందు **save** చెయ్యడం కీలకం – లేకపోతే **curr.next** మార్చాక మిగతా list పోతుంది.

```
let prev = null, curr = head;  
while (curr !== null) {  
  const next = curr.next; // ① తర్వాతిది కోల్పోకుండా save  
  curr.next = prev;      // ② pointer వెనక్కి తిప్పు  
  prev = curr;           // ③ prev ముందుకు  
  curr = next;           // ④ curr ముందుకు  
}  
return prev; // కొత్త head (పాత చివరి node)
```

4. Two-pass / two-pointer gap – "end నుండి nth", length ఆధారిత positions.

Linked List లో length ముందే తెలియదు. ఒక pass లో length లెక్కించి, రెండో pass లో target కి వెళ్ళొచ్చు (two-pass). లేదా **ఒకే pass** లో రెండు pointers మధ్య **n gap** పెట్టి, ముందున్నది end చేరేసరికి వెనకది target దగ్గర ఉండేలా చేయొచ్చు (one-pass two-pointer).

Complexity (linked list operations)

Operation	Time	గమనిక
Traverse / search	O(n)	random access లేదు, hop చేయాల్సిందే
Access k-th node	O(k)	array O(1) కాదు
Insert/delete (తెలిసిన node దగ్గర)	O(1)	pointers మార్చడమే
Reverse (in-place)	O(n) time, O(1) space	3-pointer technique

Operation	Time	గమనిక
Cycle detect / find middle	$O(n)$ time, $O(1)$ space	fast & slow

గుర్తుంచుకో: Pointer బొమ్మలు **కాగితం మీద గీసుకో**. $a.next = b$ అంటే "a నుండి బాణం b కి".
Reassignment చేసేముందు, ఏ node ఏ node ని point చేస్తుందో గీస్తే bugs 90% తగ్గుతాయి.
Interview లో కూడా బొమ్మ గీస్తూ మాట్లాడితే interviewer కి నీ thinking కనిపిస్తుంది.

6. Linked List Cycle (LeetCode #141) – Easy

సమస్య: ఒక linked list యొక్క `head` ఇస్తారు. అందులో **cycle (loop)** ఉందా చెప్పాలి. Cycle అంటే – ఏదో ఒక node యొక్క `next`, list లో **వెనక ఉన్న ఒక node** ని point చేస్తే, అప్పుడు traverse చేస్తూ పోతే ఎప్పటికీ `null` రాదు, అదే nodes చుట్టూ తిరుగుతూ ఉంటాం. `true / false` return చెయ్యి.

Constraints: nodes సంఖ్య $[0, 10^4]$; $-10^5 \leq \text{Node.val} \leq 10^5$. (Follow-up: **$O(1)$ memory** తో చెయ్యగలవా?)

ఉదాహరణ:

```

3 → 2 → 0 → -4
      ↑      |      (-4 యొక్క next తిరిగి 2 ని point చేస్తుంది)
      Output: true

1 → 2 → null      Output: false

```

ఎలా ఆలోచించాలి:

మామూలుగా list traverse చేస్తూ `null` కోసం చూస్తాం. కానీ cycle ఉంటే `null` **ఎప్పటికీ రాదు** – infinite loop! కాబట్టి "ఎప్పటికీ ఆగకపోవడం" ని ఎలా detect చేయాలి?

ఆలోచన 1: visit చేసిన nodes ని ఒక set లో పెడుతూ, మళ్ళీ అదే node వస్తే cycle. పనిచేస్తుంది కానీ $O(n)$ memory.

ఆలోచన 2 (అందమైనది): **racetrack analogy**. ఒక circular track మీద ఇద్దరు పరిగెడుతున్నారు – ఒకరు నెమ్మది (slow), ఒకరు రెట్టింపు వేగం (fast). Track circular (cycle) అయితే, fast runner చుట్టు తిరిగి slow ని **తప్పకుండా కలుస్తాడు** (lap చేస్తాడు). Track straight (cycle లేదు) అయితే, fast ముందుగా **finish line (null)** చేరుతాడు, ఎప్పటికీ కలవడు. ఇదే **Floyd's Tortoise and Hare** – $O(1)$ memory!

Brute Force / naive:

Visited nodes ని HashSet లో track చెయ్యి. ప్రతి node కి: ఇది ముందే set లో ఉందా? ఉంటే cycle → true. లేకపోతే add చేసి ముందుకు. `null` వస్తే cycle లేదు → false.

```

function hasCycleNaive(head) {
    const seen = new Set();

```

```

let curr = head;
while (curr !== null) {
  if (seen.has(curr)) return true; // మళ్ళీ అదే node → cycle
  seen.add(curr);                 // node object reference ని store
  curr = curr.next;
}
return false;
}

```

- **Time:** $O(n)$, **Space:** $O(n)$ – set లో n nodes.
- ఎందుకు "సరిపోదు": పనిచేస్తుంది, correct. కానీ follow-up **$O(1)$ memory** అడిగింది. Fast & slow తో extra space లేకుండా చేయగలం – interview లో అదే expect చేస్తారు.

Optimal Approach:

Insight (Floyd's): slow 1 step, fast 2 steps. Cycle ఉంటే ఇద్దరూ loop లోపల ఖచ్చితంగా కలుస్తారు (relative speed 1 – fast ప్రతి iteration లో slow ని 1 node దగ్గరకు చేరుతుంది, కాబట్టి miss అవ్వదు). Cycle లేకపోతే fast null చేరి loop ఆగుతుంది.

Plan:

1. slow = head, fast = head.
2. fast మరియు fast.next రెండూ non-null అయినంతవరకు loop (fast రెండు steps దూకుతుంది కాబట్టి రెండింటినీ చాక్ చెయ్యాలి):
 - slow = slow.next, fast = fast.next.next.
 - slow === fast (అదే node) అయితే → cycle → true.
3. Loop ముగిస్తే (fast null చేరింది) → cycle లేదు → false.

Solution (JavaScript):

```

function hasCycle(head) {
  let slow = head;
  let fast = head;

  // fast రెండు steps దూకుతుంది → fast, fast.next రెండూ ఉండాలి.
  while (fast !== null && fast.next !== null) {
    slow = slow.next; // నెమ్మది: 1 step
    fast = fast.next.next; // వేగం: 2 steps
    if (slow === fast) { // reference equality – అదే node object
      return true; // కలిశారు → cycle!
    }
  }

  return false; // fast, null చేరింది → అంతం ఉంది → cycle లేదు
}

```

Dry Run: 3 → 2 → 0 → -4 → (తిరిగి 2)

```

nodes: A(3) → B(2) → C(0) → D(-4) → B (cycle)
start: slow=A, fast=A

```

```

iter1: slow=B, fast=C      B===C? no
iter2: slow=C, fast=B (D→B) C===B? no      [fast: C→D then D→B]
iter3: slow=D, fast=D (B→C→D) D===D? YES → return true ✓

```

No-cycle: `1 → 2 → null` → start `slow=A, fast=A`; iter1: `slow=B, fast = A.next.next = null`; loop condition `fast !== null` fails → return false. ✓

Complexity:

- **Time:** $O(n)$ – cycle ఉంటే, slow loop లోకి ప్రవేశించాక fast ఒక్క iteration కి 1 node చొప్పున దగ్గరవుతుంది, గరిష్టంగా $O(n)$ iterations లో కలుస్తారు. Cycle లేకపోతే fast $n/2$ steps లో `null`.
- **Space:** $O(1)$ – రెండు pointers మాత్రమే. ఇదే HashSet కన్నా మెరుగు.

గుర్తుంచుకోవాల్సినది:

Fast & slow pointers – "cycle ఉందా", "middle node", "cycle మొదలు ఎక్కడ" (#142), "happy number", "duplicate number" (#287) అన్నిటికీ ఇదే. కీలక mantra: **loop condition లో fast & fast.next రెండూ చెక్ చెయ్యి**, లేకపోతే `fast.next.next` `null` మీద crash. ఇది linked-list interview లో అత్యంత common technique – గుడ్డిగా గుర్తుపెట్టుకో.

సాధారణ తప్పులు:

- **Null checks తప్పు:** `while (fast & fast.next)` – రెండూ అవసరం. `fast.next` మాత్రమే చెక్ చేసి `fast.next.next` access చేస్తే, `fast` `null` అయినప్పుడు crash.
- **Value equality vs reference:** `slow.val === fast.val` తప్పు – వేర్వేరు nodes కి same value ఉండొచ్చు. `slow === fast` (అదే object reference) చెక్ చెయ్యి.
- **Slow, fast ని ఒకేచోట కదపడం:** `slow`, `fast` ని ఒకే line లో వేర్వేరు speeds తో కదపాలి. Speeds సమానం చేస్తే ఎప్పటికీ కలవరు.
- **Empty / single node:** `head = null` లేదా ఒక్క node (`next = null`) → loop entry లోనే condition fail → false. Code సహజంగా handle చేస్తుంది, ప్రత్యేక case అవసరం లేదు.

7. Add Two Numbers (LeetCode #2) – Medium

సమస్య: రెండు non-empty linked lists ఇస్తారు – ఇవి రెండు **non-negative integers** ని represent చేస్తాయి, digits **reverse order** లో store అయి ఉంటాయి (అంటే ones digit మొదట = head). ఈ రెండు numbers ని కలిపి, sum ని **అదే reverse-order linked list** గా return చెయ్యి. ప్రతి node ఒక single digit (0-9). Leading zeros ఉండవు (number 0 తప్ప).

Constraints: ప్రతి list length `[1, 100]`; `0 <= Node.val <= 9`.

ఉదాహరణ:

```

l1 = [2,4,3] → అంటే number 342 (reverse: 2 ones, 4 tens, 3 hundreds)
l2 = [5,6,4] → అంటే number 465

```

342 + 465 = 807

Output: [7,0,8] → అంటే 807 (reverse order)

ఎలా ఆలోచించాలి:

మనం స్కూల్లో కూడిక ఎలా చేశాం? **కుడి నుండి ఎడమకు**, digit-by-digit, **carry** ని తర్వాతి column కి తీసుకెళ్తా. ఇక్కడ అదృష్టం ఏంటంటే – digits ఇప్పటికే **reverse order** లో ఉన్నాయి, అంటే **head = ones place = మనం మొదట కూడాల్సిన digit!** కాబట్టి రెండు lists ని head నుండి పక్కపక్కన నడుస్తూ, ఒక్కో స్థానం digit లు కూడి, carry maintain చేస్తూ కొత్త list కడితే సరిపోతుంది. ఇది కాగితం మీద చేసే addition ని అక్షరాలా అనుకరించడమే (simulate).

రెండు కీలక సూక్ష్మతలు: (1) lists **వేర్వేరు పాడవులు** ఉండొచ్చు – పొట్టిది అయిపోతే ఆ digit ని **0** అనుకో. (2) చివర్లో **carry మిగిలితే** ($5 + 5 = 10$) దానికీ ఒక కొత్త node కావాలి.

Brute Force / naive:

రెండు lists ని numbers గా convert చెయ్యి (traverse చేస్తూ number build), కూడు, తిరిగి digits ని list గా మార్చు.

```
// Naive – నిజ numbers గా మార్చి కూడడం. length 100 కి BREAKS!  
// list1 → 342, list2 → 465, sum=807, తిరిగి list గా. కానీ...
```

- **Time:** $O(n)$, **Space:** $O(n)$.
- **ఎందుకు సరిపోదు:** Length **100 digits** వరకు! JavaScript **Number** గరిష్ఠంగా ~ 16 digits (2^{53}) safe. 100-digit number → precision పోతుంది, తప్పు answer. **BigInt** వాడొచ్చు కానీ అది "trick" – interviewer digit-by-digit simulation చూడాలనుకుంటాడు (ఇది overflow-proof, ఏ language లోనైనా పనిచేస్తుంది).

Optimal Approach:

Insight: Elementary-school addition ని simulate చెయ్యి – head (ones) నుండి, digit-by-digit, carry తో. **Dummy head** వాడితే కొత్త list కట్టడం సులభం.

Plan:

1. **dummy** node + **current** = **dummy** (ఫలిత list కట్టడానికి). **carry** = **0**.
2. **l1** లేదా **l2** లేదా **carry** – ఏదో ఒకటి మిగిలినంతవరకు loop:
 - **x** = **l1** ? **l1.val** : **0**, **y** = **l2** ? **l2.val** : **0**.
 - **sum** = **x** + **y** + **carry**.
 - **carry** = **Math.floor(sum / 10)** (0 లేదా 1).
 - కొత్త node **sum % 10** (ones digit) → **current.next** కి attach, **current** ముందుకు.
 - **l1**, **l2** ని (ఉంటే) ముందుకు జరుపు.
3. **dummy.next** return.

carry ని loop condition లో పెట్టడం వల్ల చివరి carry (కొత్త node కావాల్సినది) automatic గా handle అవుతుంది – ప్రత్యేక code అవసరం లేదు.

Solution (JavaScript):

```
class ListNode {
  constructor(val = 0, next = null) {
    this.val = val;
    this.next = next;
  }
}

function addTwoNumbers(l1, l2) {
  const dummy = new ListNode(0); // ఫలిత list కి fake head
  let current = dummy;           // కొత్త nodes ఇక్కడ కడతాం
  let carry = 0;

  // ఒక list మిగిలినా, లేదా carry మిగిలినా continue
  while (l1 !== null || l2 !== null || carry !== 0) {
    const x = l1 ? l1.val : 0; // పొట్టి list అయిపోతే 0
    const y = l2 ? l2.val : 0;
    const sum = x + y + carry;

    carry = Math.floor(sum / 10); // 0 లేదా 1
    current.next = new ListNode(sum % 10); // ones digit → కొత్త node
    current = current.next;

    if (l1) l1 = l1.next; // ఉంటేనే ముందుకు
    if (l2) l2 = l2.next;
  }

  return dummy.next; // అసలు head (dummy తర్వాతిది)
}
```

Dry Run: l1 = [2,4,3] (342), l2 = [5,6,4] (465)

```
carry=0, dummy→(nothing)
step1: x=2,y=5,sum=2+5+0=7  carry=0  node(7)  list: 7
      l1→4, l2→6
step2: x=4,y=6,sum=4+6+0=10 carry=1  node(0)  list: 7→0
      l1→3, l2→4
step3: x=3,y=4,sum=3+4+1=8  carry=0  node(8)  list: 7→0→8
      l1→null, l2→null
loop check: l1=null, l2=null, carry=0 → stop
return [7,0,8] = 807 ✓ (342+465=807 ✓)
```

Carry-out చూద్దాం: [9,9,9,9,9,9,9] + [9,9,9,9] → చివర్లో carry=1 మిగిలితే, loop condition `carry !== 0` వల్ల ఇంకొక `node(1)` create అవుతుంది → [8,9,9,9,0,0,0,1] . ✓

Complexity:

- **Time:** $O(\max(m, n))$ – పొడవైన list ఎన్ని nodes అయితే అన్ని iterations.
- **Space:** $O(\max(m, n))$ – ఫలిత list అంత పొడవు (+1 carry కి). `dummy` తప్ప extra space లేదు.

గుర్తుంచుకోవాల్సినది:

"కొత్త linked list కట్టాలి" అంటే → **dummy head + tail pointer** pattern. `dummy` వల్ల "మొదటి node ప్రత్యేకం" అనే case పోతుంది. **Carry-in-loop-condition** trick గుర్తుంచుకో – చివరి carry ని ప్రత్యేకంగా handle చేయనవసరం లేదు. ఇదే idea: **Add Two Numbers II** (#445, forward order → stack లేదా reverse వాడు), **Multiply Strings, Plus One**, big-integer arithmetic. Digit-by-digit simulation = overflow-proof.

సాధారణ తప్పులు:

- **Number గా convert చేయడం:** 100-digit numbers కి precision పోతుంది. Digit-by-digit చెయ్యి.
- **చివరి carry మర్చిపోవడం:** `[5] + [5] = [0, 1]`. Loop condition లో `carry !== 0` లేకపోతే `[0]` వస్తుంది (తప్పు). Carry ని condition లో పెట్టు, లేదా loop తర్వాత `if (carry) current.next = new ListNode(carry)`.
- **వేర్వేరు lengths:** `l1 ? l1.val : 0` తో పొట్టి list ని `0` తో pad చెయ్యి. లేకపోతే `null.val` → crash.
- **Head ని కదపడం, dummy మర్చిపోవడం:** ఫలితం build చేసేటప్పుడు tail (`current`) ని కదుపు, `dummy` ని ముట్టొద్దు. `dummy.next` return చెయ్యి, `dummy` కాదు.

8. Merge Two Sorted Lists (LeetCode #21) – Easy

సమస్య: రెండు **sorted (ascending)** linked lists `list1`, `list2` ఇస్తారు. వాటిని ఒకే **sorted** linked list గా merge చెయ్యి. కొత్త list, ఇచ్చిన రెండు lists యొక్క **nodes ని splice (అతికించడం)** చేయడం ద్వారానే కట్టాలి (కొత్త nodes create చేయనవసరం లేదు). Merged list యొక్క head return చెయ్యి.

Constraints: ప్రతి list length `[0, 50]`; `-100 <= Node.val <= 100`; రెండూ non-decreasing order లో.

ఉదాహరణ:

```
list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]

list1 = [], list2 = [0] → Output: [0]
list1 = [], list2 = [] → Output: []
```

ఎలా ఆలోచించాలి:

రెండు sorted lists ని కలపడం – ఇది **merge sort లోని merge step** అచ్చుగుద్దినట్టు. రెండు జట్ల players ఇద్దరూ height ఆరోహణ క్రమంలో నిలబడ్డారు అనుకో. వాళ్ళను ఒకే line లో sorted గా నిలబెట్టాలి. ఎలా? రెండు జట్ల **మొదటి players ఇద్దరినీ** పోల్చి, **పోట్టివాడిని** ముందు line లోకి తీసుకో, ఆ జట్టులో తర్వాతివాడిని ముందుకు తే. మళ్ళీ compare. ఇలా ఒక జట్టు ఖాళీ అయ్యేదాకా. మిగిలిన జట్టు అంతా ఇప్పటికే sorted కాబట్టి, దాన్ని line చివర **మొత్తంగా అతికించు**.

రెండు pointers (ప్రతి list కి ఒకటి) + **dummy head** (ఫలిత list కట్టడానికి) = clean solution.

Brute Force / naive:

రెండు lists nodes అన్నీ ఒక array లోకి పోసి, array ని sort చేసి, తిరిగి list కట్టడం.

- **Time:** $O((m+n) \log(m+n))$ – sort ఖరీదు. **Space:** $O(m+n)$ array.
- **ఎందుకు సరిపోదు:** Inputs **ఇప్పటికే sorted!** ఆ property ని వాడకుండా మళ్ళీ sort చేయడం waste. Sortedness ని exploit చేస్తే **$O(m+n)$** linear, $O(1)$ extra (nodes reuse). Interviewer ఇదే చూస్తాడు.

Optimal Approach:

Insight: Two-pointer merge. రెండు heads ని పోల్చి, పొట్టిదాన్ని ఫలిత list కి link చేసి, ఆ list ను ముందుకు జరుపు. **Dummy head** వల్ల మొదటి node ప్రత్యేక case పోతుంది.

Plan:

1. `dummy` node, `tail = dummy` (ఫలిత list చివర).
2. `list1 !== null && list2 !== null` అయినంతవరకు:
 - `list1.val <= list2.val` అయితే $\rightarrow$ `tail.next = list1`, `list1 = list1.next`.
 - లేకపోతే $\rightarrow$ `tail.next = list2`, `list2 = list2.next`.
 - `tail = tail.next` (చివరను ముందుకు).
3. Loop ముగిస్తే ఒక list ఖాళీ. మిగిలినది (`list1 || list2`) ఇప్పటికే sorted $\rightarrow$ `tail.next` కి attach.
4. `dummy.next` return.

$\leq$ (కేవలం $<$ కాదు) వాడటం **stability** కి మంచిది (సమాన values లో list1 ముందు వస్తుంది).

Solution (JavaScript):

```
// ListNode: {val, next} – Linked List primer లో define చేసినదే.
function mergeTwoLists(list1, list2) {
  const dummy = new ListNode(0); // ఫలిత list కి fake head
  let tail = dummy;               // ఫలిత list యొక్క ప్రస్తుత చివర

  // రెండింటిలోనూ nodes ఉన్నంతవరకు: పొట్టిదాన్ని తీసుకో
  while (list1 !== null && list2 !== null) {
    if (list1.val <= list2.val) {
      tail.next = list1; // list1 node ని అతికించు
      list1 = list1.next; // list1 ముందుకు
    } else {
      tail.next = list2;
      list2 = list2.next;
    }
    tail = tail.next; // ఫలిత list చివరను ముందుకు
  }

  // ఒకటి ఖాళీ అయింది; మిగిలినది ఇప్పటికే sorted  $\rightarrow$  మొత్తం attach
  tail.next = list1 !== null ? list1 : list2;

  return dummy.next;
}
```

Dry Run: `list1 = [1,2,4]`, `list2 = [1,3,4]`

```

dummy→0, tail=dummy
cmp 1 vs 1: 1<=1 → tail.next=l1(1); l1→2; tail=(1)    result: 1
cmp 2 vs 1: 2<=1? no → tail.next=l2(1); l2→3; tail=(1) result: 1→1
cmp 2 vs 3: 2<=3 → tail.next=l1(2); l1→4; tail=(2)    result: 1→1→2
cmp 4 vs 3: 4<=3? no → tail.next=l2(3); l2→4; tail=(3) result: 1→1→2→3
cmp 4 vs 4: 4<=4 → tail.next=l1(4); l1→null; tail=(4) result: 1→1→2→3→4
l1=null → loop ఆగింది
tail.next = l2 (= [4]) → attach మిగిలిన 4
return [1,1,2,3,4,4] ✓

```

Complexity:

- **Time:** $O(m + n)$ – ప్రతి node ఒకసారి touch. Sort లేదు కాబట్టి linear.
- **Space:** $O(1)$ – కొత్త nodes కట్టలేదు, ఉన్న nodes నే rewire చేశాం. (dummy ఒక్క node – constant.)

గుర్తుంచుకోవాల్సినది:

Two-pointer merge = రెండు sorted sequences ని కలపడం, $O(m+n)$. ఇది **merge sort** గుండె.

Dummy head + tail pattern మళ్ళీ (కొత్త/reordered list కట్టే ప్రతి problem). Sortedness ని వాడు – sort మళ్ళీ చేయకు. Related: **Merge k Sorted Lists** (#23 – ఇదే merge ని pairwise/heap తో extend),

Sorted array merge (#88), **Interval merge**.

సాధారణ తప్పులు:

- **మిగిలిన tail attach మర్చిపోవడం:** loop ఒక list ఖాళీ అయితే ఆగుతుంది. రెండో list లో nodes మిగిలి ఉంటాయి! `tail.next = list1 || list2` తప్పనిసరి. మర్చిపోతే merged list మధ్యలోనే తెగిపోతుంది.
- **Empty lists:** `list1 = null` అయితే loop entry లోనే ఆగి, `tail.next = list2` → స్కైప్ చేసి. Dummy వల్ల ఇది ప్రత్యేక code లేకుండా పనిచేస్తుంది.
- **tail ముందుకు జరపడం మర్చిపోవడం:** ప్రతి attach తర్వాత `tail = tail.next` అవసరం. మర్చిపోతే అన్ని nodes ఒకే చోటికి link → list పాడు.
- **కొత్త nodes create చేయడం:** అవసరం లేదు (ఉన్న nodes splice చాలు). చేసినా తప్పు కాదు, కానీ $O(m+n)$ space వృథా – problem intent kill.

9. Copy List with Random Pointer (LeetCode #138) – Medium

సమస్య: ఒక special linked list ఇస్తారు – ప్రతి node లో మామూలు `next` తో పాటు ఒక **random** pointer కూడా ఉంది. `random`, list లోని **ఏ node నైనా** (లేదా `null` ని) point చేయొచ్చు. ఈ list యొక్క **deep copy** (పూర్తిగా కొత్త nodes తో నకలు) చెయ్యి. కొత్త list లోని pointers అన్నీ **కొత్త nodes ని మాత్రమే** point చేయాలి – original nodes ని కాదు. కొత్త list యొక్క head return చెయ్యి.

Constraints: nodes సంఖ్య `[0, 1000]`; `random` valid node లేదా `null`.

Node నిర్వచనం:

```
class Node {
    constructor(val, next = null, random = null) {
        this.val = val;
        this.next = next;
        this.random = random; // ఏ node నైనా (లేదా null)
    }
}
```

ఉదాహరణ:

Original: A(7) → B(13) → C(11) → D(10) → E(1)
 randoms: A.random=null, B.random=A, C.random=E, D.random=C, E.random=A
 Output: పైదానికి పూర్తి కొత్త copy – structure, randoms అన్నీ కొత్త nodes కి point.

ఎలా ఆలోచించాలి:

మామూలు list copy అయితే easy – head నుండి నడుస్తూ ప్రతి node కి కొత్త node create, `next` లు link. కానీ **random ఇక్కడ చిక్కు**, C యొక్క కొత్త copy కి `random` set చేయాలంటే, "E యొక్క కొత్త copy" కావాలి. కానీ నేను C ని process చేసేటప్పుడు E ఇంకా copy అయిందే లేదో తెలియదు (E, C తర్వాత వస్తుంది). అంటే: **"పాత node → దానికి సరిపడే కొత్త node" అనే mapping కావాలి.**

ఆలోచన 1 (సూటి): ఒక **HashMap** పెట్టు – key = పాత node, value = కొత్త node. మొదట అన్ని కొత్త nodes create చేసి map లో పెట్టు. తర్వాత రెండో pass లో, ప్రతి కొత్త node కి `next`, `random` ని map ద్వారా set చెయ్యి (పాత node యొక్క next/random ని map లో lookup చేస్తే కొత్త counterpart దొరుకుతుంది).

ఆలోచన 2 (చాకచక్యం, O(1) space): కొత్త copy ని ప్రతి original node **పక్కనే interleave** చేసి పెడితే, "పాత node యొక్క copy" = ఎప్పుడూ `original.next` – map అవసరం లేదు!

Brute Force / naive:

Map లేకుండా: ప్రతి node కి copy create చేసి `next` link చెయ్యి. తర్వాత ప్రతి node యొక్క `random` కోసం – "ఈ random original list లో ఎన్నో node?" అని head నుండి లెక్కించి (index), కొత్త list లో అదే index కి నడిచి link చెయ్యి.

- **Time:** $O(n^2)$ – ప్రతి random కి index వెతకడం $O(n)$.
- **ఎందుకు సరిపోదు:** $n=1000$ కి 10^6 – పర్వాలేదు కానీ ugly, $O(n)$ map తో సులభం. Interviewer $O(n)$ expect చేస్తాడు.

Optimal Approach 1 – HashMap ($O(n)$ time, $O(n)$ space, అర్థం చేసుకోవడం సులభం):

Insight: పాత → కొత్త mapping ని HashMap లో పెట్టు. రెండు passes: pass-1 అన్ని nodes clone చేసి map చెయ్యి; pass-2 next/random ని map lookup తో wire చెయ్యి.

Optimal Approach 2 – Interweaving ($O(n)$ time, $O(1)$ extra space, చాకచక్యం):

Insight: ప్రతి original node వెంటనే దాని clone ని `next` గా పెట్టు ($A \rightarrow A' \rightarrow B \rightarrow B' \rightarrow \dots$). అప్పుడు "X యొక్క clone" = `X.next` (map అక్కర్లేదు). Random wire చేయడం: `X'.random =`

`X.random.next` (X యొక్క random యొక్క clone). చివర్లో రెండు lists ని విడదీయి (unweave), original ని పునరుద్ధరించు.

Plan (interweaving):

1. **Pass 1 – interleave:** ప్రతి node X కి clone `X'` create, `X.next = X'`, `X'.next = పాత X.next`.
2. **Pass 2 – randoms:** ప్రతి original X కి, `X.random` ఉంటే `X.next.random = X.random.next` (అంటే `X'.random = (X.random)'`).
3. **Pass 3 – separate:** interleaved list ని రెండుగా విడదీయి – original ను తిరిగి కట్టు, copy ను వేరు చెయ్యి. Copy head return.

Solution (JavaScript) – HashMap (clean, recommended for interview):

```
function copyRandomList(head) {
    if (head === null) return null;

    const map = new Map(); // పాత node → కొత్త node

    // Pass 1: ప్రతి node కి clone create (next/random ఇంకా set చేయలేదు)
    let current = head;
    while (current !== null) {
        map.set(current, new Node(current.val));
        current = current.next;
    }

    // Pass 2: clones యొక్క next, random ని map ద్వారా wire చెయ్యి
    current = head;
    while (current !== null) {
        const clone = map.get(current);
        clone.next = current.next ? map.get(current.next) : null;
        clone.random = current.random ? map.get(current.random) : null;
        current = current.next;
    }

    return map.get(head); // కొత్త head
}
```

Solution (JavaScript) – Interweaving (O(1) extra space):

```
function copyRandomListWeave(head) {
    if (head === null) return null;

    // Pass 1: A → A' → B → B' → ... (clone ని పక్కనే జోప్పించు)
    let curr = head;
    while (curr !== null) {
        const clone = new Node(curr.val);
        clone.next = curr.next;
        curr.next = clone;
        curr = clone.next; // = పాత curr.next
    }

    // Pass 2: clone.random = curr.random యొక్క clone (= curr.random.next)
    curr = head;
    while (curr !== null) {

```

```

    if (curr.random !== null) {
        curr.next.random = curr.random.next;
    }
    curr = curr.next.next; // తర్వాతి original కి
}

// Pass 3: రెండు lists విడదీయి (original పునరుద్ధరించు, copy వేరు చెయ్యి)
curr = head;
const dummy = new Node(0);
let copyTail = dummy;
while (curr !== null) {
    copyTail.next = curr.next; // copy node
    copyTail = copyTail.next;
    curr.next = curr.next.next; // original యొక్క నిజ next పునరుద్ధరణ
    curr = curr.next;
}

return dummy.next;
}

```

Dry Run (interweaving): $A \rightarrow B$, $A.random = B$, $B.random = B$

```

Pass 1:  $A \rightarrow A' \rightarrow B \rightarrow B'$  ( $A'.val=A.val$ ,  $B'.val=B.val$ )
Pass 2:  $curr=A$ :  $A.random=B \rightarrow A'.random = B.next = B'$  ✓
        $curr=B$ :  $B.random=B \rightarrow B'.random = B.next = B'$  ✓
Pass 3: విడదీత  $\rightarrow$  original:  $A \rightarrow B$ ; copy:  $A' \rightarrow B'$ 
        $A'.random=B'$ ,  $B'.random=B'$  (అన్నీ copy nodes) ✓

```

Complexity:

- **HashMap:** Time $O(n)$ (రెండు passes), Space **$O(n)$** (map).
- **Interweaving:** Time $O(n)$ (మూడు passes), Space **$O(1)$** extra (output తప్ప – map అవసరం లేదు). చిన్న trade-off: original ని తాత్కాలికంగా mutate చేసి, తిరిగి పునరుద్ధరిస్తాం.

గుర్తుంచుకోవాల్సినది:

"కొత్త structure కట్టాలి, కానీ pointers ఒకదానికొకటి cross-reference అవుతున్నాయి" అంటే $\rightarrow$ **old $\rightarrow$ new mapping** కావాలి. సూటి పరిష్కారం **HashMap** (graph/tree clone అన్నిటికీ ఇదే – **Clone Graph** #133 కూడా). $O(1)$ కావాలంటే **interweaving trick** (clone ని original పక్కనే పెట్టి, "map ని structure లోనే encode" చేయడం). ఈ "పక్కన జొప్పించి, wire చేసి, విడదీయడం" idea అరుదైనది కానీ elegant – గుర్తుంచుకో.

సాధారణ తప్పులు:

- **Shallow copy:** కొత్త nodes కి `next / random` ని **original** nodes కి point చేయడం. అది deep copy కాదు – కొత్త nodes ని మాత్రమే point చేయాలి. (Map version లో `map.get(...)` వాడకుండా నేరుగా `current.next` పెడితే ఈ bug.)
- **`random === null` చెక్ మర్చిపోవడం:** `map.get(null) = undefined` (map లో null లేదు) – `random` ని `undefined` చేస్తుంది, `null` కాదు. `current.random ? ... : null` guard పెట్టు.

- **Interweaving లో original restore మర్చిపోవడం:** Pass 3 లో `curr.next = curr.next.next` చేయకపోతే original list శాశ్వతంగా చెడిపోతుంది (weave అలానే ఉండిపోతుంది). Copy ని వేరు చేయడంతో పాటు original ని కూడా repair చెయ్యాలి.
- **Empty list:** `head = null` → వెంటనే `return null`. లేకపోతే `head.next` → crash.

10. Reverse Linked List II (LeetCode #92) – Medium

సమస్య: ఒక linked list యొక్క `head`, మరియు రెండు integers `left`, `right` (`left <= right`) ఇస్తారు. `left` వ **position నుండి** `right` వ **position వరకు** ఉన్న nodes ని **మాత్రమే** reverse చెయ్యి (positions **1-indexed**), మిగతా list అలానే ఉంచు. మార్చిన list యొక్క head return చెయ్యి. **ఒకే pass** లో చేయగలవా (follow-up)?

Constraints: nodes సంఖ్య `[1, 500]`; `-500 <= Node.val <= 500`; `1 <= left <= right <= n`.

ఉదాహరణ:

```
head = [1,2,3,4,5], left = 2, right = 4
positions 2..4 (values 2,3,4) ని reverse → 4,3,2
Output: [1,4,3,2,5]

head = [5], left = 1, right = 1 → Output: [5] (ఏమీ మారదు)
```

ఎలా ఆలోచించాలి:

మామూలు "పూర్తి list reverse" తెలుసు (3-pointer). కానీ ఇక్కడ **మధ్యలో ఒక భాగం** మాత్రమే. రెండు సవాళ్లు: (1) reverse చేయాల్సిన segment ని కనుక్కోవడం, (2) reverse చేశాక దాన్ని ముందు భాగం, వెనుక భాగంతో సరిగ్గా **కుట్టడం (stitch)**.

Two approaches for the reversal itself:

- సులభమైన మానసిక నమూనా: `left-1` position వరకు నడిచి, అక్కడి నుండి `right` వరకు మామూలు reverse చేసి, తర్వాత మూడు కొసలు (ముందు భాగపు చివర, reversed segment, వెనుక భాగపు మొదలు) కలపడం. Bookkeeping ఎక్కువ.
- **Elegant one-pass – "head insertion":** segment ముందున్న node ని `prev` గా పట్టుకో. తర్వాత segment లోని ప్రతి node ని ఒక్కొక్కటిగా **తీసి, prev వెంటనే తర్వాత జోప్పించు** (front కి తెచ్చు). `right-left` సార్లు చేస్తే segment reverse అవుతూ, connections కూడా సహజంగా కుదురుకుంటాయి.

Dummy head వాడితే `left = 1` (అసలు head కూడా reverse లో ఉంటే) అనే edge case మాయమవుతుంది.

Brute Force / naive:

Position `left..right` values ని ఒక array లోకి copy, array ని reverse, తిరిగి ఆ positions లో values overwrite.

```
// Naive: values ని array లో పెట్టి, reverse చేసి, తిరిగి రాయడం.  
// O(n) time, O(right-left) space. కానీ pointer skill చూపదు.
```

- **Time:** $O(n)$, **Space:** $O(\text{right} - \text{left})$.
- **ఎందుకు సరిపోదు:** పనిచేస్తుంది కానీ extra space, పైగా interviewer **pointer manipulation** (in-place, $O(1)$) చూడాలనుకుంటాడు – linked list అంటేనే pointers. Values swap చేయడం "cheat".

Optimal Approach (one-pass, head insertion, $O(1)$ space):

Insight: `prev` = reverse segment ముందు node. Segment లోపలి node ని ఒక్కొక్కటిగా తీసి `prev.next` స్థానంలో ముందుకు తెచ్చు – segment తలకిందులవుతుంది.

Plan:

1. `dummy = new ListNode(0, head) . prev = dummy .`
2. `left - 1` steps నడిచి `prev` ని segment **ముందు** node కి తీసుకెళ్ళు.
3. `curr = prev.next` (reverse అయ్యే segment మొదటి node – ఇది చివరికి segment **చివర** ఉంటుంది).
4. `right - left` సార్లు "head insertion":
 - `next = curr.next` (తరలించాల్సిన node).
 - `curr.next = next.next` (curr ను next పైనండి తప్పించు).
 - `next.next = prev.next` (next ను segment మొదటికి తే).
 - `prev.next = next` (prev ను next కి link).
5. `dummy.next` return.

Solution (JavaScript):

```
// ListNode: {val, next} – primer లో define చేసినదే.  
function reverseBetween(head, left, right) {  
  const dummy = new ListNode(0, head); // head కూడా reverse లో ఉంటే safe  
  let prev = dummy;  
  
  // ① prev ని (left-1)వ node కి – అంటే reverse segment ముందు node కి తీసుకెళ్ళు  
  for (let i = 0; i < left - 1; i++) {  
    prev = prev.next;  
  }  
  
  // ② curr = segment మొదటి node. ఇది స్థిరం – reverse అయ్యాక segment చివరకు జారుతుంది.  
  const curr = prev.next;  
  
  // ③ (right-left) సార్లు: curr తర్వాతి node ని తీసి prev వెంటనే జోప్పించు (front insertion)  
  for (let i = 0; i < right - left; i++) {  
    const next = curr.next; // తరలించాల్సిన node  
    curr.next = next.next; // curr ను next పైనండి తప్పించు  
    next.next = prev.next; // next ను ప్రస్తుత segment-front ముందు పెట్టు  
    prev.next = next; // prev → next (కొత్త segment front)  
  }  
}
```

```

    return dummy.next;
}

```

Dry Run: [1,2,3,4,5], left=2, right=4

```

dummy→1→2→3→4→5. prev walk: left-1=1 step → prev=node(1)
curr = prev.next = node(2) (ఇది segment చివరకు జారుతుంది)
segment reverse iterations = right-left = 2:

iter1: next=curr.next=3
    curr.next = next.next = 4    → list: 1→2→4→5 (3 తీసేశాం), 3.next?
    next.next = prev.next = 2    → 3→2
    prev.next = next = 3         → 1→3→2→4→5
    (ఇప్పుడు: 1→3→2→4→5, curr ఇంకా node(2))

iter2: next=curr.next=4
    curr.next = next.next = 5    → 2→5
    next.next = prev.next = 3    → 4→3
    prev.next = next = 4         → 1→4→3→2→5

return 1→4→3→2→5 ✓

```

గమనిక: curr (node 2) ఎప్పుడూ కదలలేదు – segment లోని మిగతా nodes దాని ముందుకు జారాయి, అది సహజంగా segment చివరికి చేరింది, 5 కి link అయి ఉంది.

Complexity:

- **Time:** $O(n)$ – prev కి walk $O(\text{left})$, reverse $O(\text{right}-\text{left})$. మొత్తం ఒకే pass, $O(n)$.
- **Space:** $O(1)$ – pointers మాత్రమే, in-place.

గుర్తుంచుకోవాల్సినది:

Partial reverse = dummy head + segment-ముందు prev ని anchor చేయడం + head-insertion.

dummy వల్ల left=1 (head కూడా reverse) edge case పోతుంది. "Head insertion" pattern – ఒక node ని తీసి list ముందుకు తేవడం – reverse-in-place కి ప్రత్యామ్నాయ నమూనా, గుర్తుంచుకో. దీని విస్తరణ:

Reverse Nodes in k-Group (#25, తర్వాత problem), **Reverse Linked List** (#206, full).

సాధారణ తప్పులు:

- **Off-by-one (1-indexed):** positions 1-indexed. prev కి left-1 steps (segment ముందు node కి). left steps నడిస్తే ఒక node ముందుకు పోతావు – segment తప్పు.
- **curr ని కదపడం:** curr (segment మొదటి node) ని loop లో reassign చేయొద్దు. అది anchor – మిగతావి దాని చుట్టూ కదులుతాయి. దాన్ని కదిపితే pointer chaos.
- **Dummy లేకపోవడం:** left=1 అయితే అసలు head కూడా reverse అవుతుంది → head మారుతుంది. Dummy లేకపోతే ప్రత్యేక code రాయాలి. Dummy తో uniform.
- **మూడు reassignments order:** next.next = prev.next ముందు, prev.next = next తర్వాత – order తప్పితే pointer lost. కాగితం మీద గీసుకో.
- **left == right :** loop right-left = 0 సార్లు నడుస్తుంది – ఏమీ మారదు, సరైనది.

11. Reverse Nodes in k-Group (LeetCode #25) – Hard

సమస్య: ఒక linked list యొక్క `head`, ఒక integer `k` ఇస్తారు. List ని **k nodes చొప్పున groups** గా విభజించి, **ప్రతి group ని reverse** చెయ్యి. చివర్లో **k కన్నా తక్కువ nodes** మిగిలితే వాటిని **అలానే** (reverse చేయకుండా) ఉంచు. Node values మార్చకూడదు – nodes నే **rewire** చెయ్యాలి. మార్చిన list head return చెయ్యి.

Constraints: nodes సంఖ్య `[1, 5000]`; `0 <= Node.val <= 1000`; `1 <= k <= n`. (Follow-up: **O(1) extra space?**)

ఉదాహరణ:

```
head = [1,2,3,4,5], k = 2 → Output: [2,1,4,3,5]    (చివర్లో 5 ఒంటరి → అలానే)
head = [1,2,3,4,5], k = 3 → Output: [3,2,1,4,5]    (చివర్లో 4,5 < 3 → అలానే)
```

ఎలా ఆలోచించాలి:

ఇది #92 (partial reverse) ని పదేపదే చేయడం లాంటిది, కానీ **షరతుతో**: group ని reverse చేసేముందు, "ఈ group కి పూర్తి k nodes ఉన్నాయా?" చూడాలి. లేకపోతే అలానే వదిలేయాలి.

Naturally recursive గా అనిపిస్తుంది: **"మొదటి k nodes reverse చెయ్యి, తర్వాత మిగతా list కి అదే పని recursively చెయ్యి, రెండింటినీ కలుపు."** ఒక్కో group ను ఒక చిన్న sub-problem గా చూడు.

రెండు దశలు ప్రతి group కి: (1) ముందు **k nodes ఉన్నాయో లేదో లెక్కించు** (లేకపోతే group ని అలానే return). (2) ఉంటే, ఆ k nodes ని standard 3-pointer తో reverse చేసి, reversed group యొక్క పాత-మొదలు (ఇప్పుడు కొత్త-చివర) ని **తర్వాతి recursive result** కి కలుపు.

Brute Force / naive:

Group లోని values ని array లోకి copy, reverse, తిరిగి rewrite – group group గా. లేదా nodes ని stack లో push చేసి k దగ్గర pop చేస్తూ కొత్త list కట్టడం.

```
// Naive (stack): k nodes push, తర్వాత pop చేస్తూ link → group reversed.
// తక్కువ మిగిలితే వాటిని అలానే append. O(n) time కానీ O(k) space.
```

- **Time:** $O(n)$, **Space:** $O(k)$ (stack లేదా array).
- **ఎందుకు "సరిపోదు":** పనిచేస్తుంది. కానీ follow-up **O(1) space** అడిగింది. In-place pointer reversal తో సాధ్యం. (Recursion version $O(n/k)$ call-stack వాడుతుంది; నిజమైన $O(1)$ కావాలంటే iterative. రెండూ కింద చర్చిస్తాం.)

Optimal Approach (recursive, clean):

Insight: ప్రతి group ని recursively handle చెయ్యి – "k ఉన్నాయా చూడు → ఉంటే reverse → తోక ని recursive-rest కి కలుపు".

Plan:

1. `head` నుండి **k nodes ఉన్నాయా** చూడు (k steps నడుపు). లేకపోతే `head` ని అలానే return (చివరి పాజీక group).
2. ఉంటే, standard 3-pointer తో **మొదటి k nodes reverse** చెయ్యి. Reverse తర్వాత `prev` = కొత్త group head, `curr` = (k+1)వ node = తర్వాతి group మొదలు.
3. ఇప్పుడు పాత `head` (reverse అయ్యాక ఈ group **చివరి** node) యొక్క `next` ని – `reverseKGroup(curr, k)` (మిగతా list recursively) కి link చెయ్యి.
4. `prev` (కొత్త head) return.

Solution (JavaScript):

```
// ListNode: {val, next} – primer లో define చేసినదే.
function reverseKGroup(head, k) {
  // ① ముందుగా k nodes ఉన్నాయా చూడు
  let node = head;
  for (let i = 0; i < k; i++) {
    if (node === null) return head; // k కన్నా తక్కువ → అలానే వదిలేయి
    node = node.next;
  }
  // ఇక్కడ node = (k+1)వ node (తర్వాతి group మొదలు)

  // ② మొదటి k nodes ని reverse చెయ్యి (standard 3-pointer)
  let prev = null;
  let curr = head;
  for (let i = 0; i < k; i++) {
    const next = curr.next; // save
    curr.next = prev;      // వెనక్కి తీపు
    prev = curr;           // prev ముందుకు
    curr = next;           // curr ముందుకు
  }
  // ఇప్పుడు: prev = ఈ group కొత్త head; curr = తర్వాతి group మొదలు (=(k+1)వ)
  // పాత head ఇప్పుడు ఈ group చివరి node → దాన్ని మిగతా list కి కలుపు

  // ③ head.next = మిగతా list ను recursively reverse చేసినది
  head.next = reverseKGroup(curr, k);

  // ④ కొత్త head return
  return prev;
}
```

Dry Run: [1,2,3,4,5], k=2

```
call reverseKGroup(1→2→3→4→5, 2)
k-check: 2 nodes ఉన్నాయా? 1,2 ✓ (node లెక్క తర్వాత node=3)
reverse first 2:
  start prev=null, curr=1
  i0: next=2; 1.next=null; prev=1; curr=2
  i1: next=3; 2.next=1; prev=2; curr=3
  → group: 2→1 ; prev=2(కొత్త head); curr=3(తర్వాతిది); head=1(group చివరి)
head(1).next = reverseKGroup(3→4→5, 2)
  | k-check: 3,4 ✓ (node=5)
  | reverse: 4→3 ; prev=4; curr=5; head=3
  | head(3).next = reverseKGroup(5, 2)
  |   | k-check: 5 ✓, తర్వాత node=null → k రెండోది కి null → return 5 (అలానే)
```

```

└ 3.next = 5 → group: 4→3→5
└ return 4 → (3→4→5 భాగం ఫలితం: 4→3→5)
1.next = 4 → 2→1→4→3→5
return 2 (అసలు కొత్త head)
Final: [2,1,4,3,5] ✓

```

Complexity:

- **Time:** $O(n)$ – ప్రతి node k-check లో ఒకసారి, reverse లో ఒకసారి touch → $O(n)$ మొత్తం.
- **Space:** $O(n/k)$ – recursion call stack (ప్రతి group కి ఒక frame). **నిజమైన $O(1)$** కావాలంటే iterative version రాయాలి (group-tail ను track చేస్తూ ఒక outer loop; అదే logic, recursion లేకుండా). Interview లో recursive చెప్పి, " $O(1)$ కావాలంటే iterative గా మార్చిచ్చు" అని చెప్పడం సరిపోతుంది.

గుర్తుంచుకోవాల్సినది:

"k nodes చొప్పున ఏదైనా చెయ్యి" = **check-k-then-process, recursively** (లేదా **iterative with group boundaries**). ముందు **feasibility check** (k ఉన్నాయా), తర్వాత **process**, తర్వాత **recurse on rest** – ఇది chunked linked-list problems కి template. #92 (partial reverse) ఇక్కడ building block. Related: **Swap Nodes in Pairs** (#24 = ఇదే k=2), **rotate in groups**.

సాధారణ తప్పులు:

- **k-check మర్చిపోవడం:** k కన్నా తక్కువ మిగిలిన group ని కూడా reverse చేస్తే తప్పు ([1,2,3,4,5] , k=2 లో 5 ని reverse చేయకూడదు). Reverse **ముందు** k nodes ఉన్నాయో లెక్కించు.
- **తర్వాతి group తో connection:** పాత head reverse తర్వాత group **చివరి** node అవుతుంది. దాని next ని recursive result కి link చేయడం **కీలకం** – మర్చిపోతే groups తెగిపోతాయి, list break.
- **ఏ node return చేయాలి:** reverse తర్వాత **prev** (కొత్త group head) return, head (ఇప్పుడు group tail) కాదు.
- **next save మర్చిపోవడం:** 3-pointer reverse లో `const next = curr.next` **మొదట** చెయ్యి, `curr.next` మార్చకముందే. లేకపోతే మిగతా list పోతుంది.

12. Remove Nth Node From End of List (LeetCode #19) – Medium

సమస్య: ఒక linked list యొక్క head , ఒక integer n ఇస్తారు. **చివరి నుండి n వ node** ని తీసేసి (remove), మార్చిన list యొక్క head return చెయ్యి. (n valid – $1 \leq n \leq \text{list length}$.) **ఒకే pass** లో చేయగలవా (follow-up)?

Constraints: nodes సంఖ్య [1, 30] ; $0 \leq \text{Node.val} \leq 100$; $1 \leq n \leq \text{sz}$.

ఉదాహరణ:

```

head = [1,2,3,4,5], n = 2 → చివరి నుండి 2వది = 4 → తీసేయి → [1,2,3,5]
head = [1], n = 1 → [] (ఏకైక node తీసేశాం)

```

head = [1,2], n = 1 → [1] (చివరిది తీసేసాం)

ఎలా ఆలోచించాలి:

"చివరి నుండి nవది" – linked list లో **వెనక నుండి లెక్కించడం** (back pointers లేవు). సూటి ఆలోచన: మొదటి pass లో length **L** లెక్కించు, అప్పుడు "చివరి నుండి nవది" = "మొదటి నుండి $(L - n + 1)$ వది". రెండో pass లో దాని **ముందు** node కి వెళ్లి, pointer bypass చెయ్యి. ఇది **two-pass**, పనిచేస్తుంది.

మరింత చక్కని ఆలోచన (**one-pass**): రెండు pointers మధ్య **n nodes gap** పెట్టు. **fast** ని ముందు **n steps** జరుపు. తర్వాత **fast**, **slow** రెండింటినీ కలిసి జరుపు – **fast** చివరి node చేరేసరికి, **slow** ఖచ్చితంగా **తీసేయాల్సిన node కి ముందు** ఉంటుంది (gap స్థిరం n కాబట్టి). అప్పుడు **slow.next = slow.next.next**.

Dummy head కీలకం – head నే తీసేయాల్సిన case ($n = \text{length}$) ని ప్రత్యేక code లేకుండా handle చేస్తుంది.

Brute Force / naive (two-pass):

Pass 1: length **L** లెక్కించు. Pass 2: $L - n$ steps నడిచి (dummy నుండి), ఆ node యొక్క **next** ని bypass.

- **Time:** $O(L)$ (రెండు passes అయినా $O(2L) = O(L)$), **Space:** $O(1)$.
- ఎందుకు "సరిపోదు": correct, కానీ list ని రెండుసార్లు traverse చేస్తుంది. Follow-up **one-pass** అడిగింది. Two-pointer gap తో ఒకే pass సాధ్యం – interviewer అది expect చేస్తాడు. (Streaming data / list ని ఒకసారే చదవగలిగే situation లో one-pass అవసరం.)

Optimal Approach (one-pass, two-pointer gap):

Insight: **fast** ను **slow** కన్నా **n nodes ముందు** ఉంచు. తర్వాత ఇద్దరూ కలిసి కదిలితే, **fast** చివరకు చేరేసరికి **slow** target-ముందు node వద్ద.

Plan:

1. **dummy = new ListNode(0, head)**. **fast = dummy**, **slow = dummy**.
2. **fast** ను **n steps** ముందుకు జరుపు (ఇప్పుడు **fast**, **slow** మధ్య **n gap**).
3. **fast.next !== null** అయినంతవరకు, **fast**, **slow** రెండింటినీ ఒక్కో step జరుపు. (**fast.next** null అయితే **fast** చివరి node – ఆగు.)
4. ఇప్పుడు **slow** = తీసేయాల్సిన node కి **ముందు** node. **slow.next = slow.next.next** (bypass).
5. **dummy.next** return.

Solution (JavaScript):

```
// ListNode: {val, next} – primer లో define చేసినదే.
function removeNthFromEnd(head, n) {
  const dummy = new ListNode(0, head); // head తీసేసే case కి safe
  let fast = dummy;
  let slow = dummy;

  // ① fast ను n steps ముందుకు → fast, slow మధ్య n gap
```

```

for (let i = 0; i < n; i++) {
    fast = fast.next;
}

// ② ఇద్దరినీ కలిసి జరుపు; fast చివరి node చేరేదాకా
while (fast.next !== null) {
    fast = fast.next;
    slow = slow.next;
}
// ఇప్పుడు slow = తీసేయాల్సిన node కి ముందు node

// ③ target ను bypass చెయ్యి
slow.next = slow.next.next;

return dummy.next;
}

```

Dry Run: [1,2,3,4,5] , n=2 (చివరి నుండి 2వది = 4)

```

dummy→1→2→3→4→5. fast=slow=dummy
① fast ను 2 steps: fast=dummy→1→2 (fast=node 2)
② while fast.next != null:
    fast=3, slow=1
    fast=4, slow=2
    fast=5, slow=3
    fast.next == null (5 చివరిది) → ఆగు
    slow = node(3) ← ఇది తీసేయాల్సిన 4 కి ముందు ✓
③ slow.next = slow.next.next → 3.next = 5 (4 ను bypass)
list: 1→2→3→5
return [1,2,3,5] ✓

```

Head-removal case: [1,2] , n=2 → fast 2 steps: dummy→1→2 (fast=node 2). fast.next = null → while skip. slow=dummy. slow.next = slow.next.next → dummy.next = node(2) → head ఇప్పుడు 2 → [2] ... ఆగు, n=2 అంటే చివరి నుండి 2వది = 1, తీస్తే [2]. ✓ Dummy వల్ల head removal పనిచేసింది.

Complexity:

- **Time:** $O(L)$ – ఒకే pass (fast మొత్తం list ఒకసారి traverse).
- **Space:** $O(1)$ – రెండు pointers.

గుర్తుంచుకోవాల్సినది:

"చివరి నుండి nవది" / "fixed gap" = two pointers with n-gap, one pass. ఒక pointer ను n ముందు జరిపి, తర్వాత lockstep – ఇది linked-list లో "వెనక నుండి" positions కి universal trick. **Dummy head** వల్ల head-removal edge case మాయం. Related: **middle of list** (gap బదులు speed difference), **nth from end**, sliding-window-on-list ideas.

సాధారణ తప్పులు:

- **Head removal (dummy లేకపోవడం):** $n = \text{length}$ అయితే head నే తీయాలి. Dummy లేకపోతే slow కి "ముందు node" ఉండదు → ప్రత్యేక code. Dummy తో uniform, bug-free.

- **Gap off-by-one:** `fast` ను `n` steps జరిపి, `while (fast.next !== null)` (`fast` చివరి node దగ్గర ఆగాలి, `null` కాదు). `n+1` steps లేదా `while (fast !== null)` వాడితే `slow` తప్పు స్థానంలో ఆగుతుంది – పక్క node తీసేస్తావు.
- **Fast ను ఎక్కువ జరపడం:** loop condition `fast.next !== null` – `slow` ను **target-ముందు** ఆపుతుంది. `fast !== null` వాడితే `slow` target మీదకే వెళ్ళి, తప్పు node తీసేస్తుంది.
- **`slow.next` null చెక్:** `n` valid (constraint) కాబట్టి `slow.next` ఎప్పుడూ ఉంటుంది – safe. కానీ `n` invalid రావొచ్చు అనుకుంటే guard చేర్చు.

13. Remove Duplicates from Sorted List II (LeetCode #82) – Medium

సమస్య: ఒక **sorted** linked list యొక్క `head` ఇస్తారు. **Duplicate values ఉన్న nodes అన్నిటినీ తీసి** – original list లో **distinct (ఏకైక) numbers మాత్రమే** మిగలాలి. (గమనిక: #83 లో duplicate ని ఒకటి ఉంచుతాం; ఇక్కడ #82 లో duplicate ఉన్న value ని **పూర్తిగా** తీసేస్తాం.) Sorted order maintain చేస్తూ head return చెయ్యి.

Constraints: nodes సంఖ్య `[0, 300]` ; `-100 <= Node.val <= 100` ; list **sorted (ascending)**.

ఉదాహరణ:

```
head = [1,2,3,3,4,4,5] → 3, 4 duplicate → రెండింటినీ తీసి → [1,2,5]
head = [1,1,1,2,3] → 1 duplicate → అన్ని 1 తీసి → [2,3]
```

ఎలా ఆలోచించాలి:

List **sorted** – కాబట్టి duplicates ఎప్పుడూ **పక్కపక్కనే** (adjacent) ఉంటాయి. ఇది కీలకం: ఒక value duplicate అవునా కాదా తెలుసుకోవడానికి కేవలం **తర్వాతి node** ని చూస్తే చాలు (`curr.val === curr.next.val` ?).

రెండు cases: (1) `curr` value దాని తర్వాతిదానితో సమానం అయితే → ఇది duplicate group మొదలు → **ఆ value ఉన్న nodes అన్నిటినీ skip** చెయ్యి, `prev` ని group తర్వాతిదానికి link చెయ్యి. (2) సమానం కాకపోతే → `curr` unique → `prev` ని ముందుకు జరుపు.

కీలక subtlety: duplicate group ని skip చేసేటప్పుడు, `prev` ను **కదపకూడదు** – ఎందుకంటే మనం `prev` తర్వాత ఏం వస్తుందో ఇంకా ఖాయం చేయలేదు (తర్వాత వచ్చేది కూడా duplicate అవ్వొచ్చు). `prev` ను stable గా ఉంచి, దాని `next` ని మారుస్తూ ఉంటాం.

Dummy head అవసరం – మొదటి node(s) నే duplicate అయితే (`[1,1,2]`) head మారుతుంది.

Brute Force / naive:

Pass 1: ప్రతి value ఎన్నిసార్లు వచ్చిందో HashMap లో count. Pass 2: count == 1 ఉన్న nodes తో కొత్త list కట్టు (లేదా count > 1 వాటినీ filter).

- **Time:** $O(n)$, **Space:** $O(n)$ (count map). (Sorted కాకపోయినా పనిచేస్తుంది.)
- ఎందుకు "సరిపోదు": పనిచేస్తుంది, కానీ list **already sorted** – duplicates adjacent. ఆ property వాడితే **$O(1)$ space** (map అవసరం లేదు, పక్కపక్క compare చాలు). Interviewer sortedness ని exploit చేయమంటాడు.

Optimal Approach ($O(1)$ space, sortedness వాడి):

Insight: Sorted కాబట్టి duplicates adjacent. `prev` (చివరి confirmed-unique node) ను anchor చేసి, `curr` తో scan. Duplicate group కనిపిస్తే ఆ value అంతా skip చేసి `prev.next` ను దాటించు.

Plan:

1. `dummy = new ListNode(0, head)` . `prev = dummy` , `curr = head` .
2. `curr !== null` అయినంతవరకు:
 - `curr.next` ఉండి `curr.val === curr.next.val` అయితే (duplicate group):
 - `dupVal = curr.val` . `curr.val === dupVal` అయినంతవరకు `curr = curr.next` (group మొత్తం skip).
 - `prev.next = curr` (group ని bypass; **prev కదలదు**).
 - లేకపోతే (unique node):
 - `prev = curr` (prev ముందుకు – ఇది confirmed unique).
 - `curr = curr.next` .
3. `dummy.next` return.

Solution (JavaScript):

```
// ListNode: {val, next} – primer లో define చేసినదే.
function deleteDuplicates(head) {
  const dummy = new ListNode(0, head); // మొదటి nodes నే duplicate అయితే safe
  let prev = dummy; // చివరి confirmed-unique node (anchor)
  let curr = head;

  while (curr !== null) {
    // curr ఒక duplicate group మొదలా?
    if (curr.next !== null && curr.val === curr.next.val) {
      const dupVal = curr.val;
      // ఆ value ఉన్న nodes అన్నిటినీ skip
      while (curr !== null && curr.val === dupVal) {
        curr = curr.next;
      }
      // మొత్తం group ని bypass. prev ను కదపాద్దు (తర్వాతిది కూడా dup అవ్వచ్చు)
      prev.next = curr;
    } else {
      // unique node → prev ను ఇక్కడికి జరుపు
      prev = curr;
      curr = curr.next;
    }
  }

  return dummy.next;
}
```

Dry Run: [1,2,3,3,4,4,5]

```
dummy→1→2→3→3→4→4→5. prev=dummy, curr=1
curr=1: 1.next=2, 1≠2 → unique. prev=1, curr=2
curr=2: 2.next=3, 2≠3 → unique. prev=2, curr=3
curr=3: 3.next=3, 3==3 → dup group! dupVal=3
        skip: curr=3(2nd), curr.val==3 → curr=4
        prev.next = curr(4) → 2→4 (ఠెండు 3లు bypass). prev ఇంకా 2
curr=4: 4.next=4, 4==4 → dup group! dupVal=4
        skip: curr=4(2nd) → curr=5
        prev.next = curr(5) → 2→5 (ఠెండు 4లు bypass). prev ఇంకా 2
curr=5: 5.next=null → unique. prev=5, curr=null
loop అగింది.
list: 1→2→5
return [1,2,5] ✓
```

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి visit (inner while కూడా overall $O(n)$, amortized).
- **Space:** $O(1)$ – pointers మాత్రమే. HashMap version కన్నా మెరుగు.

గుర్తుంచుకోవాల్సినది:

Sorted list $\Rightarrow$ duplicates adjacent $\Rightarrow$ పక్కపక్క compare, $O(1)$ space. "Anchor `prev` at last-known-good, advance `curr` to scan, bypass bad runs" – filter-in-place pattern. `prev` ను confirmed-unique దగ్గర ఉంచడం కీలకం (duplicate skip చేసేటప్పుడు కదపాద్దు). Related: **#83** (duplicate ఒకటి ఉంచు – logic కొంచెం సులభం), **remove elements** (#203), sorted-array dedup (#26/#80).

సాధారణ తప్పులు:

- **Duplicate skip చేసేటప్పుడు `prev` కదపడం:** [1,2,2,3] లో 2 group skip చేశాక, తర్వాత వచ్చేది 3. కానీ [1,2,2,2] లాంటిది అయితే? `prev` ను 2 కి జరిపేస్తే తప్పు – `prev` ను duplicate node కి ఎప్పుడూ జరపాద్దు. `prev.next` ను మార్చు, `prev` ను stable గా ఉంచు.
- **#82 vs #83 గందరగోళం:** #83 duplicate value ని ఒకటి ఉంచుతుంది; #82 పూర్తిగా తీసేస్తుంది. ఇక్కడ #82 – group మొత్తం bypass చెయ్యి.
- **Dummy లేకపోవడం:** [1,1,2] లో head నే తీయాలి $\rightarrow$ head మారుతుంది. Dummy లేకపోతే ప్రత్యేక handling. Dummy తో uniform.
- **`curr.next` null చెక్:** `curr.val == curr.next.val` రాయడానికి ముందు `curr.next != null` చూడాలి – లేకపోతే చివరి node దగ్గర `null.val` $\rightarrow$ crash.

14. Rotate List (LeetCode #61) – Medium

సమస్య: ఒక linked list యొక్క `head`, ఒక integer `k` ఇస్తారు. List ని కుడివైపు (right) `k` స్థానాలు rotate చెయ్యి. అంటే చివరి `k` nodes ని ముందుకు తెచ్చి, మిగతా వాటిని వెనక్కి జరపడం. Rotate చేసిన list head return చెయ్యి.

Constraints: nodes సంఖ్య $[0, 500]$; $-100 \leq \text{Node.val} \leq 100$; $0 \leq k \leq 2 \times 10^9$ (**k చాలా పెద్దది కావచ్చు!**).

ఉదాహరణ:

```
head = [1,2,3,4,5], k = 2 → కుడికి 2 → చివరి 2 (4,5) ముందుకు → [4,5,1,2,3]
head = [0,1,2], k = 4 →  $4 \% 3 = 1$  → కుడికి 1 → [2,0,1]
```

ఎలా ఆలోచించాలి:

మొదట గమనిక: **k** list length కన్నా చాలా పెద్దది కావచ్చు (2×10^9 !). కానీ length **L** ఉన్న list ని **L** సార్లు rotate చేస్తే **మళ్ళీ మొదటికి** వస్తుంది. కాబట్టి అసలు rotations = $k \% L$. ఇది మొదట లెక్కించకపోతే time-out (2 billion iterations!).

ఇప్పుడు rotate ఎలా? కుడికి **k** rotate అంటే – **చివరి k nodes** ముందుకు వస్తాయి. అంటే కొత్త list యొక్క **break point:** కొత్త head = చివరి నుండి kవ node = మొదటి నుండి $(L - k)$ వ node. కొత్త tail = దాని ముందు node.

అందమైన trick: list ని **circular** చేసి (చివరి node ను head కి link), తర్వాత సరైన చోట **మళ్ళీ తెంపు**. Break point వద్ద తెంపితే rotation పూర్తి.

Brute Force / naive:

$k \% L$ సార్లు, ఒక్కో rotation చెయ్యి: ప్రతిసారీ చివరి node కనుక్కుని, దాన్ని తీసి head ముందు పెట్టు.

- **Time:** $O(k \times L)$ worst case (ప్రతి rotation $O(L)$ చివరి node కనుక్కోవడానికి). $k \% L$ వాడకపోతే $O(k \times L)$ = catastrophic.
- **ఎందుకు సరిపోదు:** k పెద్దది అయితే నెమ్మది. $k \% L$ + circular trick తో **ఒకే pass $O(L)$** లో అవుతుంది.

Optimal Approach (circular list trick, $O(L)$):

Insight: List ను ring గా మార్చి (tail → head), కొత్త break point $(L - k \% L)$ వద్ద తెంపు.

Plan:

1. Edge: head null, లేదా single node, లేదా $k == 0 \rightarrow$ అలానే return.
2. **Length L + tail** కనుక్కో (list చివరి దాకా నడుపు).
3. $k = k \% L$. $k == 0 \rightarrow$ rotation అవసరం లేదు, head return.
4. **Circular** చెయ్యి: tail.next = head.
5. కొత్త tail = $(L - k)$ వ node (head నుండి $L - k - 1$ steps, 1-indexed లో $L-k$ వ node వద్ద). కొత్త head = newTail.next.
6. newTail.next = null (ring తెంపు). కొత్త head return.

Solution (JavaScript):

```
// ListNode: {val, next} – primer లో define చేసినదే.
function rotateRight(head, k) {
  // ① trivial cases
```

```

if (head === null || head.next === null || k === 0) return head;

// ② length + tail కనుక్కో
let length = 1;
let tail = head;
while (tail.next !== null) {
    tail = tail.next;
    length++;
}

// ③ అసలు rotations
k = k % length;
if (k === 0) return head; // పూర్తి చుట్టు → మారదు

// ④ ring గా మార్పు
tail.next = head;

// ⑤ కొత్త tail = (length - k)వ node (head నుండి length-k-1 steps)
let stepsToNewTail = length - k;
let newTail = head;
for (let i = 1; i < stepsToNewTail; i++) {
    newTail = newTail.next;
}

// ⑥ ring ను break point వద్ద తెంపు
const newHead = newTail.next;
newTail.next = null;

return newHead;
}

```

Dry Run: [1,2,3,4,5] , k=2

```

① trivial కాదు
② length: 1→2→3→4→5, length=5, tail=node(5)
③ k = 2 % 5 = 2
④ ring: 5.next = 1 → 1→2→3→4→5→(1)...
⑤ stepsToNewTail = 5 - 2 = 3
    newTail walk: i=1→node(2), i=2→node(3). loop ఆగింది (i<3).
    newTail = node(3)
⑥ newHead = newTail.next = node(4)
    newTail.next = null → 3.next = null
    list: 4→5→1→2→3
return [4,5,1,2,3] ✓

```

[0,1,2] , k=4 : length=3, k=4%3=1, ring 2→0, stepsToNewTail=3-1=2, newTail walk i=1→node(1), newTail=node(1), newHead=node(2), 1.next=null → [2,0,1] . ✓

Complexity:

- **Time:** $O(L)$ – length కి ఒక pass, newTail కి ఒక partial pass. $k \% L$ వల్ల k పెద్దదైనా $O(L)$.
- **Space:** $O(1)$ – pointers మాత్రమే.

గుర్తుంచుకోవాల్సినది:

Rotation $\Rightarrow$ (1) $k \% L$ తో redundant rotations తీసేయి, (2) circular చేసి break point వద్ద తెంపు.

ఈ "ring + re-cut" idea rotate problems కి signature (arrays #189 కూడా – reverse-based కానీ మనసు అదే). $k \% L$ మర్చిపోవడం అత్యంత common bug – k పెద్దదిగా ఇచ్చి interviewer ఇదే test చేస్తాడు. Length కనుక్కోవడం, tail పట్టుకోవడం రెండూ ఒకే pass లో చెయ్యి.

సాధారణ తప్పులు:

- $k \% L$ మర్చిపోవడం: $k = 2 \times 10^9$ కి k సార్లు rotate చేస్తే time-out. Modulo తప్పనిసరి. $k \% L == 0$ అయితే మారదు – ముందే return.
- కుడి vs ఎడమ rotation గందరగోళం: ఇది right rotation. కొత్త head = $(L-k)$ వ node. k వ node అనుకుంటే (left rotation) తప్పు.
- Off-by-one (newTail walk): newTail = $(L-k)$ వ node కావాలంటే head నుండి $L-k-1$ steps (i.e., for $i=1; i < L-k$). Steps లెక్క తప్పితే పక్క చేట తెంపుతావు.
- Ring break చేయకపోవడం: newTail.next = null మర్చిపోతే list circular గా ఉండిపోతుంది $\rightarrow$ traversal infinite loop. తెంపడం తప్పనిసరి.
- Empty / single node: head / head.next null అయితే rotate అనవసరం $\rightarrow$ ముందే return (length loop crash నివారణ కూడా).

15. Partition List (LeetCode #86) – Medium

సమస్య: ఒక linked list యొక్క head, ఒక value x ఇస్తారు. List ని rearrange చెయ్యి – x కన్నా చిన్న values ఉన్న nodes అన్నీ, x కన్నా పెద్ద-లేదా-సమాన values ఉన్న nodes అన్నిటికీ ముందు వచ్చేలా. రెండు partitions లోపల, nodes యొక్క original relative order ని కాపాడాలి (stable). Rearranged list head return చెయ్యి.

Constraints: nodes సంఖ్య $[0, 200]$; $-100 \leq \text{Node.val} \leq 100$; $-200 \leq x \leq 200$.

ఉదాహరణ:

```
head = [1,4,3,2,5,2], x = 3
< 3: 1,2,2 (original order)  >= 3: 4,3,5 (original order)
Output: [1,2,2,4,3,5]

head = [2,1], x = 2  $\rightarrow$  < 2: 1 ; >= 2: 2  $\rightarrow$  [1,2]
```

ఎలా ఆలోచించాలి:

ఇది quicksort లోని partition step ని గుర్తుతెస్తుంది – pivot x చుట్టూ elements ని రెండు వైపులా విభజించడం. కానీ ఇక్కడ stability (original order) కావాలి, ఇది array-swap partition ఇవ్వదు.

Linked list superpower: nodes ని physically అటాచటా కదపనవసరం లేదు – కొత్త pointers తో re-link చేస్తే చాలు. కాబట్టి plan: రెండు వేర్వేరు lists కట్టు – ఒకటి $< x$ (less), ఇంకొకటి $\geq x$ (greater). Original list ని ఒకసారి traverse చేస్తూ, ప్రతి node ని దాని value ప్రకారం సరైన list చివర append చెయ్యి.

Original order లో traverse చేస్తూం కాబట్టి stability సహజంగా వస్తుంది. చివర్లో **less list చివర** → **greater list మొదలు** కి కలుపు.

రెండు lists కి రెండు **dummy heads** వాడితే append గొడవ లేకుండా clean.

Brute Force / naive:

Nodes/values ని array లోకి తీసి, stable partition (values < x ముందు, >= x తర్వాత, order కాపాడుతూ) చేసి, తిరిగి list కట్టు. లేదా in-place లో node ని కావాల్సిన చోటికి తరలించడానికి పదేపదే search – $O(n^2)$.

- **Time:** $O(n)$ (array version), **Space:** $O(n)$.
- ఎందుకు "సరిపోదు": Array version పనిచేస్తుంది కానీ extra space, పైగా linked-list యొక్క re-linking strength ని వాడదు. Two-list approach **$O(1)$ extra** (nodes reuse), single pass – cleaner, interviewer-preferred.

Optimal Approach (two dummy lists, stable, $O(1)$ space):

Insight: రెండు separate chains కట్టు – **less** (< x), **greater** ($\geq$ x). ఒకే pass, original order → stable. చివర్లో join.

Plan:

1. రెండు dummy nodes: **lessDummy**, **greaterDummy**. Tail pointers: **less = lessDummy**, **greater = greaterDummy**.
2. **curr = head**; curr ఉన్నంతవరకు:
 - **curr.val < x** → **less.next = curr**; **less = less.next**.
 - లేకపోతే → **greater.next = curr**; **greater = greater.next**.
 - **curr = curr.next**.
3. **greater.next = null** – greater list చివరను ముగించు (**కీలకం:** పాత node లో ఇంకా పాత **next** ఉంది, cycle రావొచ్చు).
4. **less.next = greaterDummy.next** – less list చివరను greater list మొదలికి కలుపు.
5. **lessDummy.next** return.

Solution (JavaScript):

```
// ListNode: {val, next} – primer లో define చేసినదే.
function partition(head, x) {
  const lessDummy = new ListNode(0); // < x nodes చైన్
  const greaterDummy = new ListNode(0); // >= x nodes చైన్
  let less = lessDummy; // less చైన్ tail
  let greater = greaterDummy; // greater చైన్ tail

  let curr = head;
  while (curr !== null) {
    if (curr.val < x) {
      less.next = curr; // less చైన్ కి append
      less = less.next;
    } else {
      greater.next = curr; // greater చైన్ కి append
      greater = greater.next;
    }
    curr = curr.next;
  }
  greater.next = null;
  return less.next;
}
```

```

    }
    curr = curr.next;
}

// greater చివరను ముగించు - పాత next ఉంటే cycle వస్తుంది!
greater.next = null;
// less చివర → greater మొదలు
less.next = greaterDummy.next;

return lessDummy.next;
}

```

Dry Run: [1,4,3,2,5,2], x=3

```

lessDummy→0 (less tail), greaterDummy→0 (greater tail)
curr=1: 1<3 → less: 1 ; less=node(1)
curr=4: 4<3? no → greater: 4 ; greater=node(4)
curr=3: 3<3? no → greater: 4→3 ; greater=node(3)
curr=2: 2<3 → less: 1→2 ; less=node(2)
curr=5: 5<3? no → greater: 4→3→5 ; greater=node(5)
curr=2: 2<3 → less: 1→2→2 ; less=node(2, 2nd)
curr=null → ఆగు

greater.next = null → greater ఛైన్: 4→3→5→null
less.next = greaterDummy.next (=4) → less ఛైన్: 1→2→2→4→3→5
return [1,2,2,4,3,5] ✓ (< 3 order 1,2,2 ✓ ; >= 3 order 4,3,5 ✓)

```

Complexity:

- **Time:** $O(n)$ – ఒకే pass, ప్రతి node ఒకసారి.
- **Space:** $O(1)$ – కొత్త nodes లేవు (dummies తప్ప constant), ఉన్న nodes నే re-link.

గుర్తుంచుకోవాల్సినది:

"Nodes ని categories గా విభజించి, order కాపాడాలి" = ప్రతి category కి ఒక dummy-headed chain, ఒకే pass, చివర్లో join. ఈ "multiple buckets, each a linked chain, then concatenate" pattern చాలా విస్తృతం: **Odd Even Linked List (#328)**, **sort list by parity**, stable bucketing. Linked list లో "physically కదపడం" అవసరం లేదు – re-link. **చివర్లో last chain యొక్క next = null** మర్చిపోవద్దు (cycle నివారణ).

సాధారణ తప్పులు:

- **greater.next = null మర్చిపోవడం:** అత్యంత common, ప్రమాదకర bug. చివరి greater node లో దాని పాత next ఇంకా ఉంటుంది – అది less/greater లోని ముందు node ని point చేస్తూ **cycle** సృష్టిస్తుంది → infinite loop. Explicit గా **null** చేయాలి.
- **Stability పోగొట్టడం:** original order లో traverse చేస్తే, ప్రతి chain లో nodes సహజంగా original relative order పొందుతాయి. Swap-based array partition వాడితే stability పోతుంది.
- **< vs <= :** x కి సమానం ఉన్న nodes **greater (>= x)** side కి వెళ్ళాలి. **curr.val < x** (strict) less కి. **<=** వాడితే x సమానం less లోకి వెళ్ళి తప్పు.

- **Two dummies లేకపోవడం:** dummies లేకుండా, ప్రతి chain మొదటి node ను ప్రత్యేకంగా handle చేయాలి (chain ఖాళీగా ఉందా చెక్). Dummies తో append uniform, bug-free.

16. LRU Cache (LeetCode #146) – Medium

సమస్య: ఒక **LRU (Least Recently Used) Cache** design చెయ్యి – fixed **capacity** తో. రెండు operations, **రెండూ $O(1)$** లో:

- **get(key)** – key ఉంటే దాని value return చెయ్యి, మరియు ఆ key ను "**most recently used**" గా **mark** చెయ్యి. లేకపోతే **-1**.
- **put(key, value)** – key ఉంటే value update, లేకపోతే కొత్తగా insert. Insert తర్వాత cache **capacity** దాటితే, **least recently used** entry ను **evict** (తీసేయి). ఇది కూడా most-recently-used అవుతుంది.

Constraints: $1 \leq \text{capacity} \leq 3000$; $0 \leq \text{key}, \text{value} \leq 10^4$; get / put కలిపి 2×10^5 calls; **రెండూ average $O(1)$** కావాలి.

ఉదాహరణ:

```
LRUCache cache = new LRUCache(2); // capacity 2
put(1,1); put(2,2);
get(1);    → 1      (ఇప్పుడు order: 2 [old] , 1 [recent])
put(3,3);  → capacity దాటింది → LRU (=2) evict.  cache: {1, 3}
get(2);    → -1     (2 తీసేశాం)
put(4,4);  → LRU (=1) evict.  cache: {3, 4}
get(1);    → -1
get(3);    → 3
get(4);    → 4
```

ఎలా ఆలోచించాలి:

రెండు అవసరాలు, రెండూ $O(1)$:

1. **key తో value ను వెంటనే వెతకడం** → ఇది **HashMap** పని ($O(1)$ lookup).
2. "**ఏది least/most recently used**" అనే **order** ను **maintain** చేయడం, ఏ **entry** నైనా **$O(1)$** లో "**చివరికి/ముందుకి**" జరపడం, **LRU** ను **$O(1)$** లో **తీసేయడం** → ఇక్కడ చిక్కు. Array అయితే మధ్య నుండి తీసి చివరకు జరపడం $O(n)$ (shifting). కావాల్సింది: ఏ node నైనా **$O(1)$** లో **unlink** చేసి, **$O(1)$** లో ఒక చివరకు జరపగల structure. అదే **Doubly Linked List (DLL)**! Node కి **prev**, **next** రెండూ ఉంటే, ఆ node ను దాని pointers తో $O(1)$ లో తప్పించవచ్చు.

కీలక combo: **HashMap** (key → DLL node) + **Doubly Linked List** (recency order). Map $O(1)$ lookup ఇస్తుంది; DLL $O(1)$ reorder/evict ఇస్తుంది. Convention: **head వైపు = most recent, tail వైపు = least recent**. Access అయిన node ను head కి జరుపు; evict అవసరమైతే tail ముందున్న node తీసేయి.

Sentinel (dummy) **head** మరియు **tail** nodes వాడితే null checks (list ఖాళీ, మొదటి/చివరి node) అన్నీ మాయం.

Brute Force / naive:

Array/list లో `[key, value, lastUsedTime]` పెట్టు. `get / put` కి array scan చేసి entry వెతుకు; evict కి minimum timestamp entry వెతుకు.

- **Time:** $O(n)$ per operation (scan).
- **ఎందుకు సరిపోదు:** Problem స్పష్టంగా **$O(1)$** అడిగింది. 2×10^5 calls $\times O(n) \rightarrow$ time-out. HashMap + DLL తో నిజంగా $O(1)$.

Optimal Approach 1 – HashMap + Doubly Linked List (interview-expected, structure నువ్వే కడతావు):

Insight: Map `key  $\rightarrow$  node` ; DLL recency order. Access $\rightarrow$ node ను front కి; evict $\rightarrow$ tail-మొందు node.

Plan: sentinel `head / tail`. Helper `_remove(node)` ($O(1)$ unlink), `_addToFront(node)` (head తర్వాత చొప్పించు).

- `get(key)` : లేకపోతే `-1`. ఉంటే node ను `_remove` + `_addToFront` (most recent), `node.val` return.
- `put(key, value)` : ఉంటే val update + front కి జరుపు. లేకపోతే `size == capacity` అయితే `tail.prev` (LRU) ను `_remove` + map నుండి delete; కొత్త node create, `_addToFront`, map లో పెట్టు.

Optimal Approach 2 – JavaScript Map (Map insertion order ను గుర్తుంచుకుంటుంది!):

Insight: JS Map **insertion order preserve** చేస్తుంది, `map.keys().next().value` = అత్యంత పాత `key = O(1)`. Access అయినప్పుడు `delete` + `set` చేస్తే ఆ key చివరికి (most recent) జరుగుతుంది. DLL ను manually కట్టినవసరం లేదు – Map ఆ పని చేస్తుంది. Concise, కానీ interview లో "నీకు DLL వచ్చా" చూడాలంటే Approach 1 రాయాలి.

Solution (JavaScript) – HashMap + DLL (interview version):

```
class DLLNode {
  constructor(key = 0, val = 0) {
    this.key = key; // eviction లో map నుండి delete చేయడానికి key కావాలి
    this.val = val;
    this.prev = null;
    this.next = null;
  }
}

class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.map = new Map(); // key  $\rightarrow$  DLLNode
    // sentinel head & tail – null checks తప్పించడానికి
    this.head = new DLLNode(); // head.next = most recent
  }
}
```

```

        this.tail = new DLLNode(); // tail.prev = least recent (evict candidate)
        this.head.next = this.tail;
        this.tail.prev = this.head;
    }

    _remove(node) {          // O(1) unlink
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    _addToFront(node) {      // head వెంటనే తర్వాత చొప్పించు = most recent
        node.next = this.head.next;
        node.prev = this.head;
        this.head.next.prev = node;
        this.head.next = node;
    }

    get(key) {
        if (!this.map.has(key)) return -1;
        const node = this.map.get(key);
        this._remove(node);    // ప్రస్తుత స్థానం నుండి తీసి
        this._addToFront(node); // most recent గా జరుపు
        return node.val;
    }

    put(key, value) {
        if (this.map.has(key)) {
            // ఉన్న key: value update + front కి
            const node = this.map.get(key);
            node.val = value;
            this._remove(node);
            this._addToFront(node);
            return;
        }
        // కొత్త key: capacity దాటితే LRU (tail.prev) evict
        if (this.map.size >= this.capacity) {
            const lru = this.tail.prev;
            this._remove(lru);
            this.map.delete(lru.key); // అందుకే node లో key store చేశాం
        }
        const node = new DLLNode(key, value);
        this._addToFront(node);
        this.map.set(key, node);
    }
}

```

Solution (JavaScript) – JS Map (concise version):

```

class LRUCache {
    constructor(capacity) {
        this.capacity = capacity;
        this.map = new Map(); // insertion order = recency (front=old, back=recent)
    }

    get(key) {
        if (!this.map.has(key)) return -1;
        const value = this.map.get(key);
    }
}

```

```

    this.map.delete(key); // తీసి...
    this.map.set(key, value); // మళ్ళీ పెడితే చివరికి (most recent) జరుగుతుంది
    return value;
}

put(key, value) {
    if (this.map.has(key)) {
        this.map.delete(key); // reorder కోసం తీసి
    } else if (this.map.size >= this.capacity) {
        // అత్యంత పాత key (Map లో మొదటిది) = LRU → evict
        const oldestKey = this.map.keys().next().value;
        this.map.delete(oldestKey);
    }
    this.map.set(key, value); // most recent గా (చివర) పెట్టు
}
}

```

Dry Run: capacity=2; put(1,1), put(2,2), get(1), put(3,3), get(2)

```

(DLL version. head ≠ ... ≠ tail ; head వైపు = recent)
put(1,1): map{1}; DLL: head≠1≠tail
put(2,2): map{1,2}; DLL: head≠2≠1≠tail (2 recent, 1 old)
get(1) : map has 1 → remove 1, addFront 1 → head≠1≠2≠tail ; return 1
        (ఇప్పుడు 2 = LRU)
put(3,3): కొత్త key; size 2 == capacity 2 → evict tail.prev = 2
        map delete 2; కొత్త 3 addFront → head≠3≠1≠tail ; map{1,3}
get(2) : map has 2? no → return -1 ✓

```

Order సరిగ్గా track అయింది – get(1) వల్ల 1 recent అయి, 2 evict అయింది (naive గా 1 కాదు).

Complexity:

- **Time:** get , put రెండూ **O(1)** average – Map lookup O(1), DLL unlink/insert O(1).
- **Space:** O(capacity) – map లో గరిష్టంగా capacity entries + అన్ని DLL nodes.

గుర్తుంచుకోవాల్సినది:

"O(1) lookup + O(1) ordering/eviction" = HashMap + Doubly Linked List. ఇది system-design-flavored, SSE interviews లో అత్యంత favorite. **Doubly** linked ఎందుకు? – ఏ node నైనా **O(1)** లో **unlink** చేయాలంటే దాని prev కావాలి. **Sentinel head/tail** nodes null-check headache తీసేస్తాయి. Node లో **key ను కూడా** store చెయ్యి (evict చేసేటప్పుడు map నుండి delete చేయడానికి). ఇదే combo: **LFU Cache** (#460, కొంచెం కష్టం), **All O(1) data structure** (#432), in-memory cache design (Redis/Memcached mental model). JS లో Map shortcut ఉన్నా, **interview లో DLL version రాయడం మంచిది** – data-structure depth చూపిస్తుంది.

సాధారణ తప్పులు:

- **Node లో key store చేయకపోవడం:** evict చేసేటప్పుడు DLL నుండి node తీశాం, కానీ map.delete(?) కి key కావాలి. Node లో key లేకపోతే map లో stale entry మిగిలిపోతుంది (memory leak + తప్పు lookups). Node కి key field తప్పనిసరి.

- **get లో recency update మర్చిపోవడం:** get కేవలం value return చేయకూడదు – ఆ key ను **most recent గా mark** చేయాలి (`remove` + `addToFront`). మర్చిపోతే LRU order తప్పు, తప్పు eviction.
- **put existing key లో reorder + capacity రెండూ:** ఉన్న key ను update చేసినప్పుడు కూడా most-recent కి జరపాలి; పైగా అది **కొత్త insert కాదు** కాబట్టి evict చేయకూడదు (size పెరగలేదు). రెండు branches విడిగా handle చెయ్యి.
- **Singly linked list వాడటం:** singly లో ఒక node ను unlink చేయాలంటే దాని **ముందు node** కావాలి → $O(n)$ search. **Doubly** తప్పనిసరి $O(1)$ కి.
- **Sentinel లేకుండా:** head/tail sentinels లేకపోతే – list ఖాళీ, ఏకైక node, head/tail evict – అన్న ప్రత్యేక null checks. Sentinels తో అన్న uniform.
- **JS Map iteration తప్పు:** `map.keys().next().value` = **మొదటి (పాత)** key. దీన్ని మర్చిపోయి తప్పు end evict చేస్తే LRU కి బదులు MRU తీసేస్తావు.

ముగింపు: Pattern Cheat-Sheet (interview ముందు ఒకసారి చదువు)

ఈ 16 problems వెనుక ఉన్నది కొన్ని core patterns మాత్రమే. Problem statement లో ఈ **signals** కనిపిస్తే, ఏ pattern వాడాలో వెంటనే గుర్తుపట్టు:

Signal (problem లో కనిపించేది)	Pattern	ఉదాహరణ problems
Matching / nesting / balanced / valid	Stack (LIFO)	Valid Parentheses (#20), Simplify Path (#71)
Parse / evaluate / "చివరి క్లింప్ చేసి combine"	Stack	RPN (#150), Basic Calculator (#224)
$O(1)$ లో min/max కావాలి	Auxiliary (parallel) stack	Min Stack (#155)
Cycle ఉందా / middle node	Fast & Slow pointers	Linked List Cycle (#141)
కొత్త / reordered list కట్టాలి	Dummy head + tail pointer	Add Two Numbers (#2), Merge (#21), Partition (#86)
In-place reverse (పూర్తి / భాగం / groups)	3-pointer reversal / head-insertion	Reverse II (#92), k-Group (#25)
"చివరి నుండి నవది" / fixed gap	Two pointers, n-gap, one pass	Remove Nth From End (#19)
Sorted list లో dedup / filter	Adjacent compare, prev anchor	Remove Duplicates II (#82)
Rotate / cyclic shift	k % L + circular + re-cut	Rotate List (#61)
Cross-referencing pointers ని clone	HashMap old→new / interweave	Copy Random List (#138)

Signal (problem లో కనిపించేది)	Pattern	ఉదాహరణ problems
$O(1)$ lookup + $O(1)$ ordering/eviction	HashMap + Doubly Linked List	LRU Cache (#146)

అన్నిటికీ వర్తించే 5 బంగారు నియమాలు:

1. **head** ని ఎప్పుడూ కదపాద్దు – ఒక temp pointer తో traverse చెయ్యి. Head పోతే list పోతుంది.
2. **Head మారొచ్చు/పోవచ్చు అనిపిస్తే dummy head వాడు** – edge cases అన్నీ మాయం.
3. **Pointer reassign చేసేముందు next ని ఒక temp లో save చెయ్యి** – లేకపోతే మిగతా list కోల్పోతావు.
4. **Fast & slow loop condition: while (fast && fast.next)** – రెండూ చెక్ చెయ్యకపోతే crash.
5. **కాగితం మీద బొమ్మ గీసుకో** – nodes, arrows. Interview లో బొమ్మ గీస్తూ మాట్లాడితే thinking కనిపిస్తుంది, bugs తగ్గుతాయి.

గుర్తుంచుకో: DSA అంటే బట్టి కాదు – **pattern recognition + pointer discipline**. ఈ 11 patterns తెలిస్తే, LeetCode 150 లోని Stack/Linked List problems మాత్రమే కాదు, వీటి variants వందల్లో solve చేయగలవు. ప్రతి problem ని "ఏ signal → ఏ pattern" అని అనువదించడం అలవాటు చేసుకో. అదే SSE interview లో గెలిపిస్తుంది.